Chapter 4

The Processor

Introduction

- CPU performance factors
  - Instruction count
    - Determined by ISA and compiler
  - CPI and Cycle time
    - Determined by CPU hardware
- We will examine two LEGv8 implementations
  - A simplified version
  - A more realistic pipelined version
- Simple subset, shows most aspects
  - Memory reference: LDUR, STUR
  - Arithmetic/logical: `add`, `sub`, `and`, `or`, `slt`
  - Control transfer: `beq`, `j`

ETAPAS DE UNA INSTRUC

- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

The Main Control Unit

- Control signals derived from instruction

Field	opcode	Rm	shamt	Rn	Rd
Bit positions	31:21	20:16	15:10	9:5	4:0

a. R-type instruction

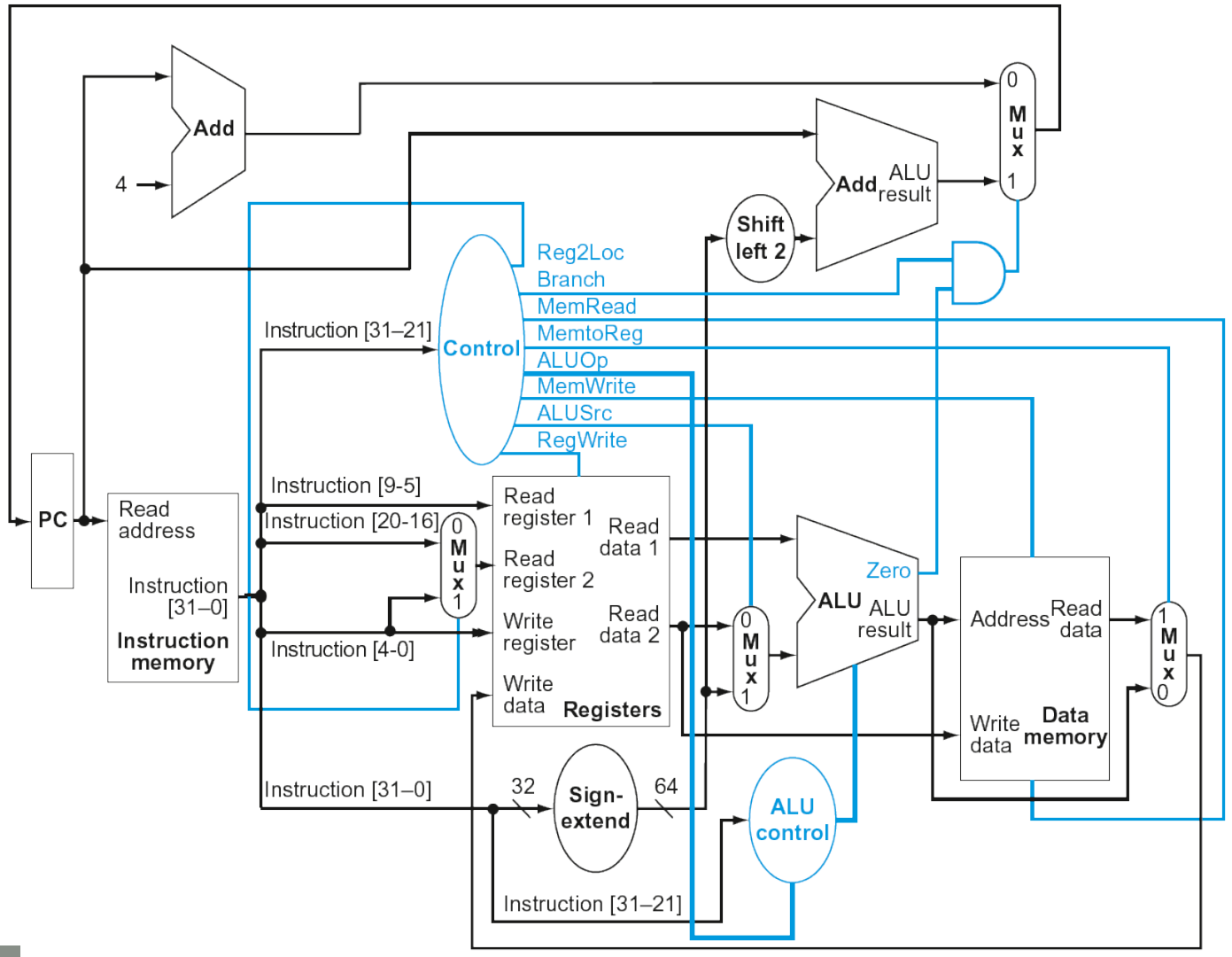
Field	1986 or 1984	address	0	Rn	Rt
Bit positions	31:21	20:12	11:10	9:5	4:0

b. Load or store instruction

Field	180	address	Rt
Bit positions	31:26	23:5	4:0

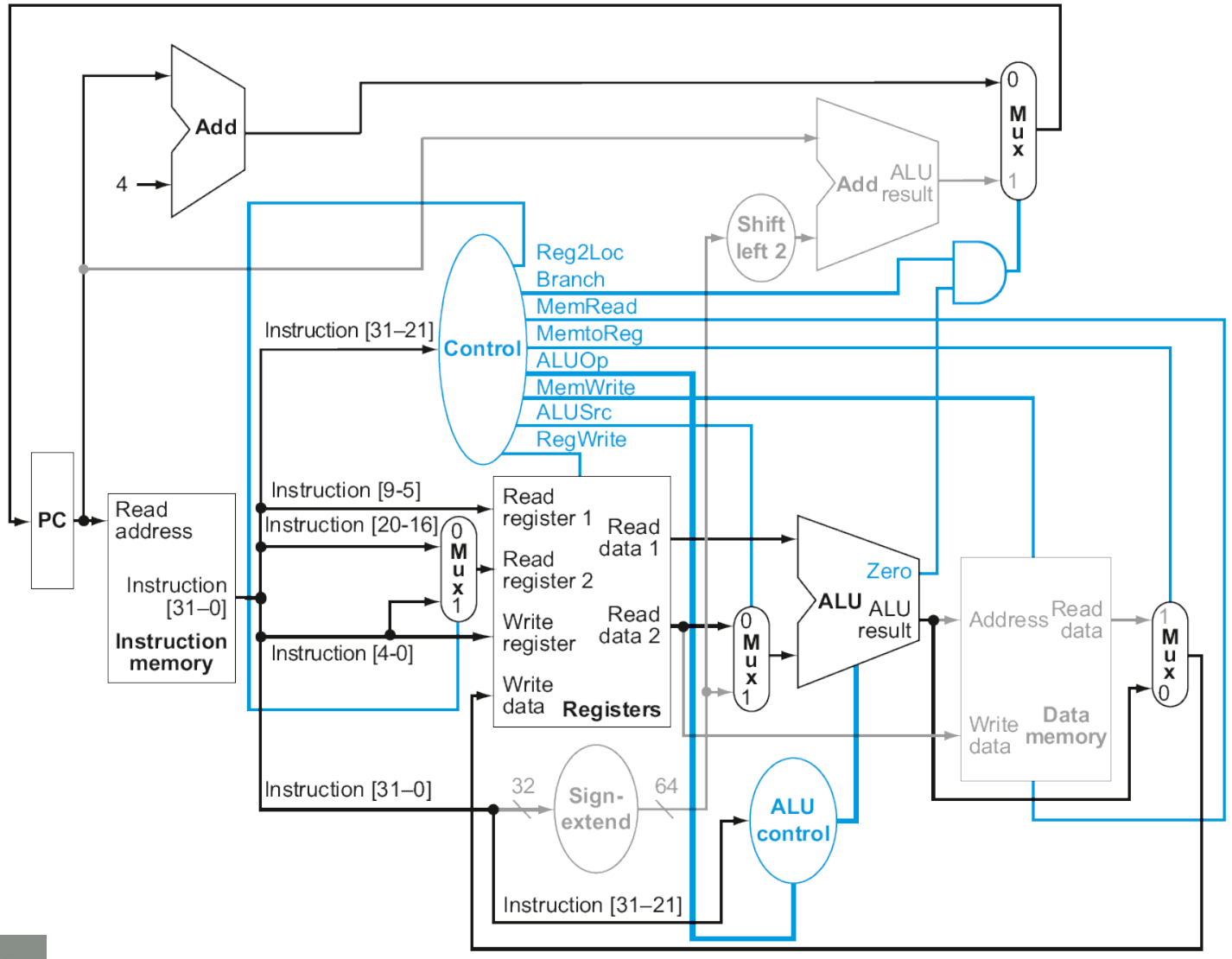
c. Conditional branch instruction

Datapath With Control



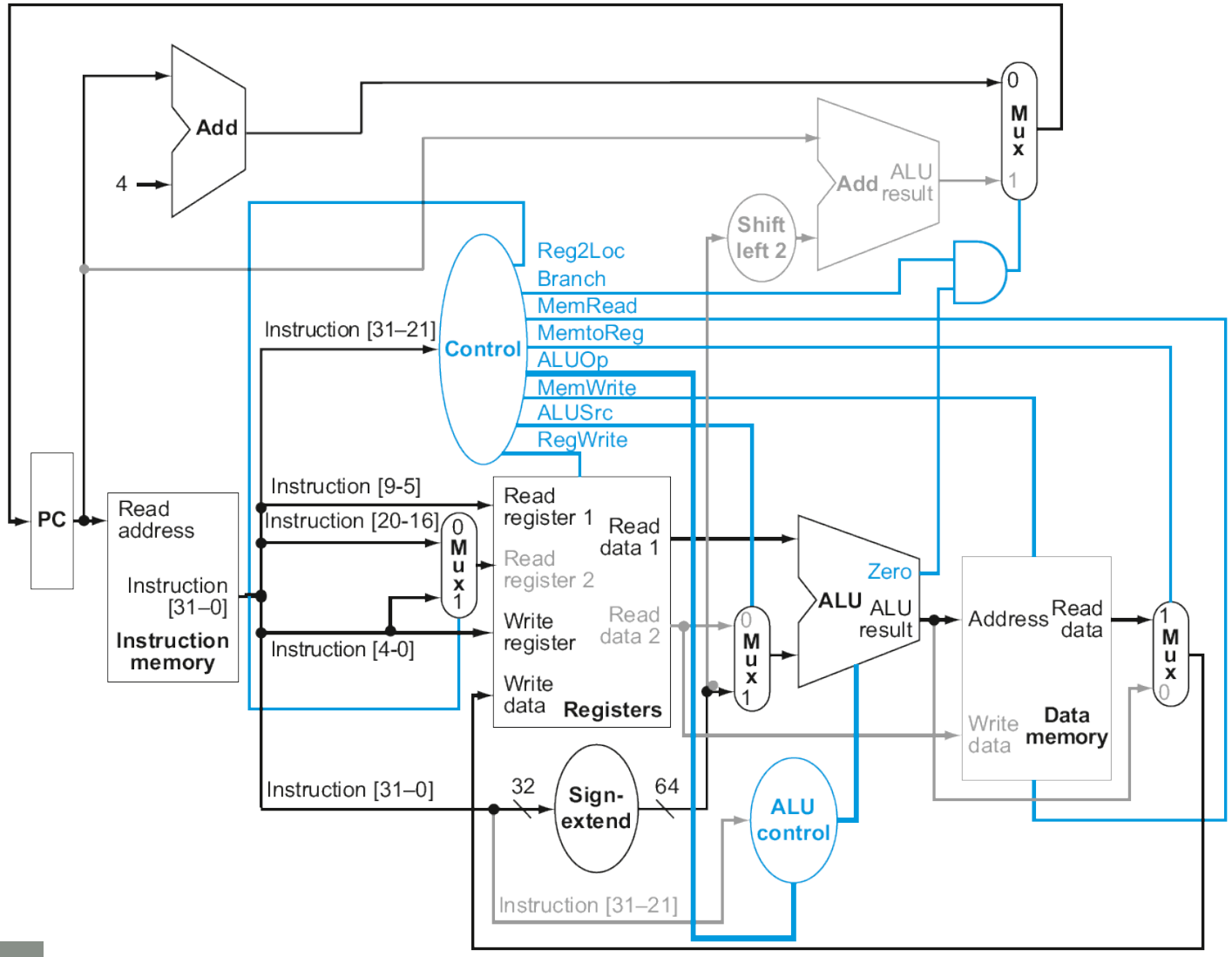
Datapath With Control

R-Type Instruction



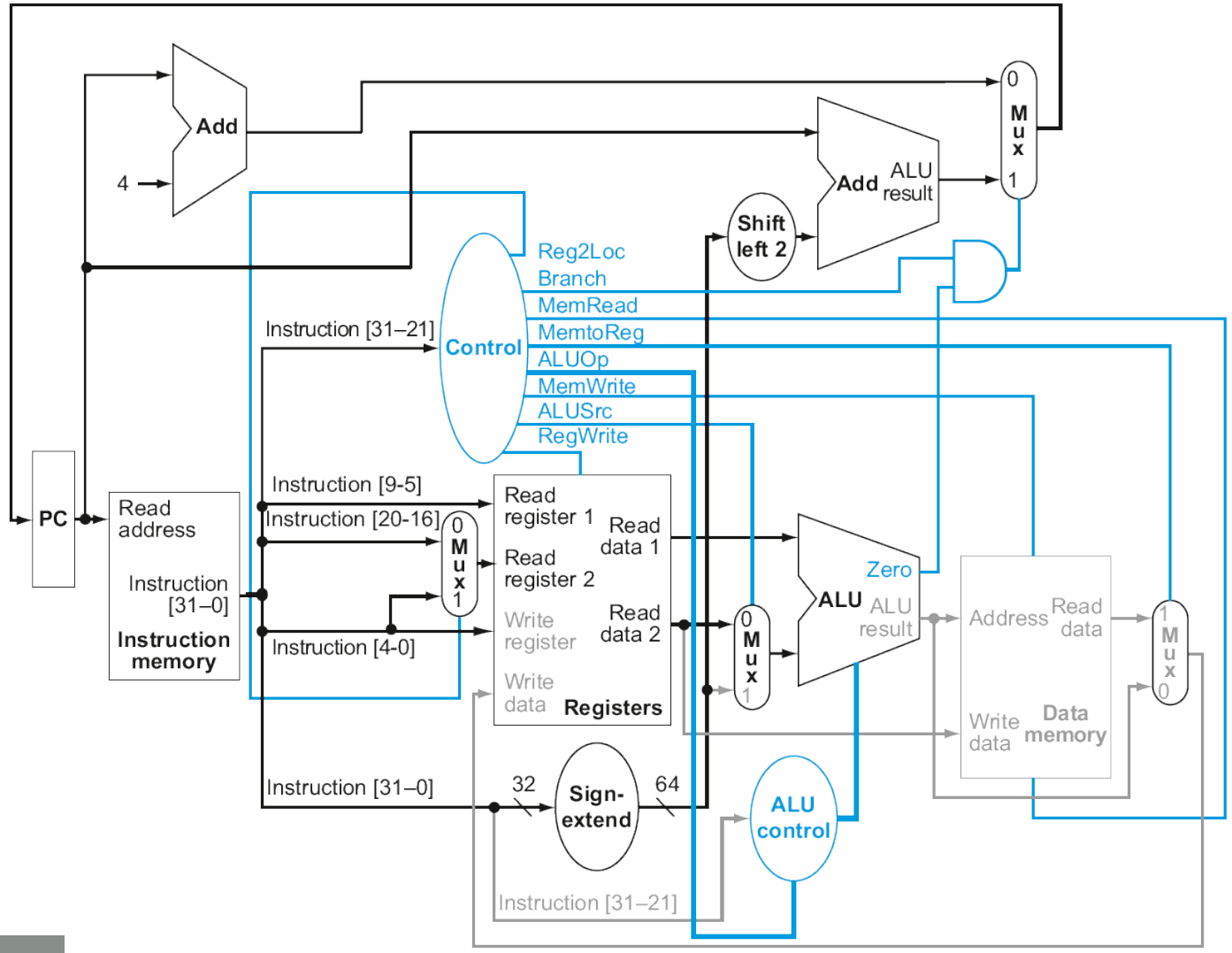
R-Type Instruction

Load Instruction



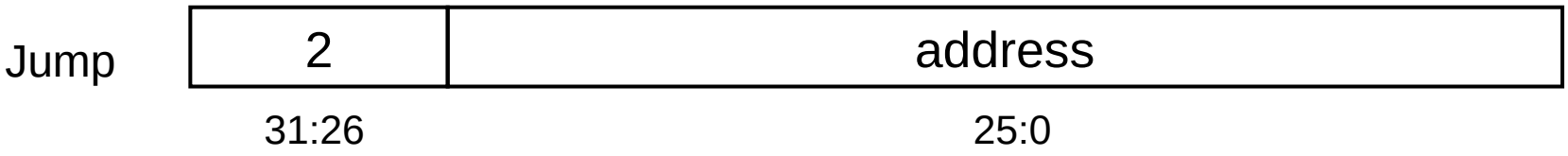
Load Instruction

CBZ Instruction



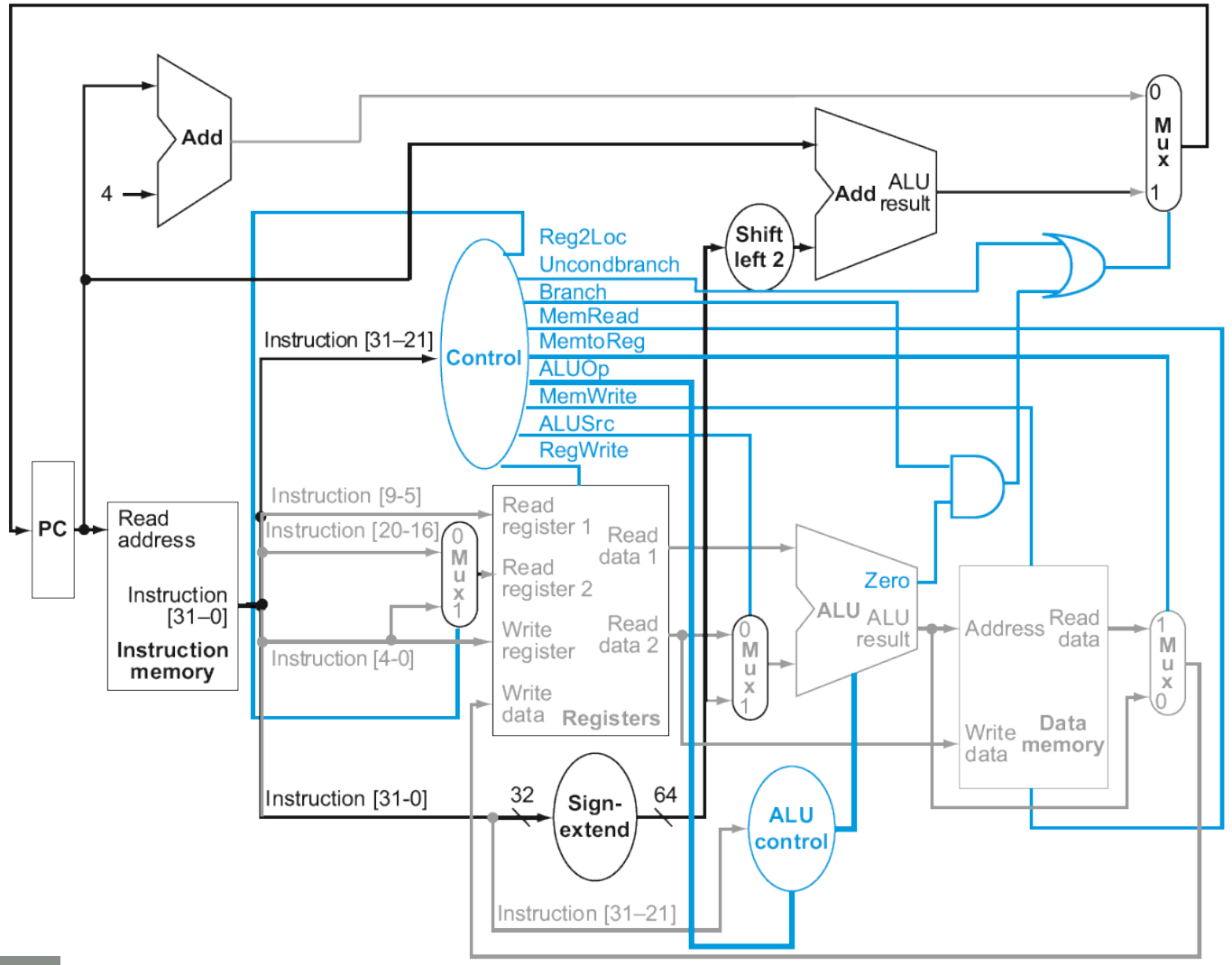
CBZ Instruction

Implementing Uncnd'l Branch



- Jump uses word address
- Update PC with concatenation of
  - Top 4 bits of old PC
  - 26-bit jump address
  - 00
- Need an extra control signal decoded from opcode

Datapath With B Added



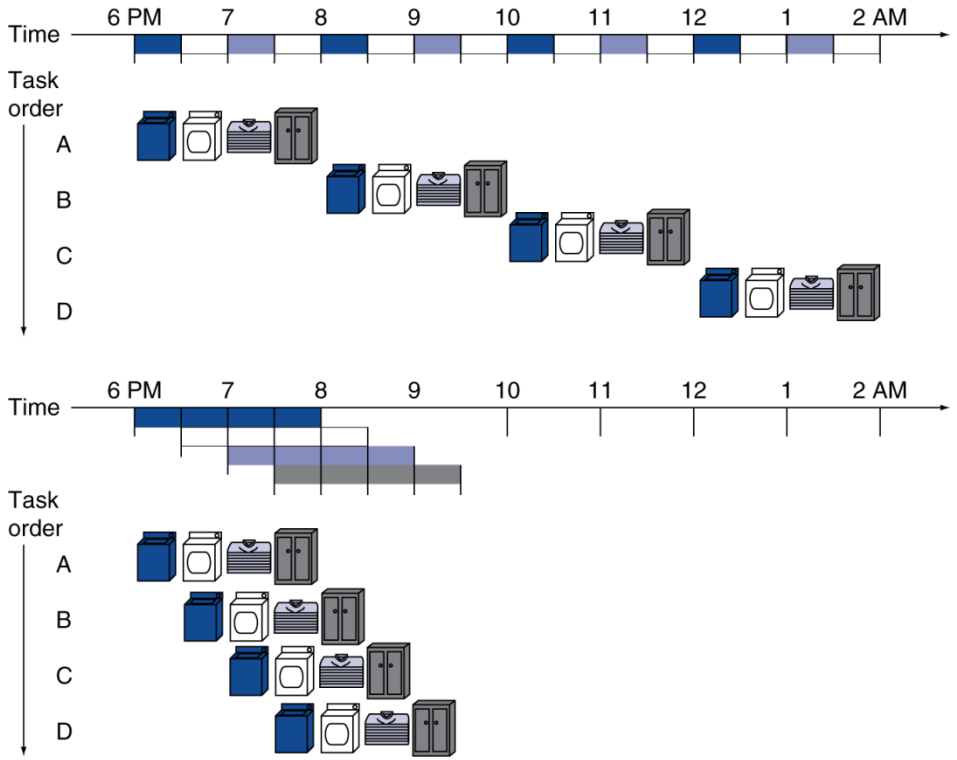
Datapath With B Added

Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining

Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance



- Four loads:
  - Speedup
    = 8/3.5 = 2.3
- Non-stop:
  - Speedup
    = 2n/0.5n + 1.5 ≈ 4
    = number of stages

Pipelining Analogy

DEDUCIR FORMULA GENERAL DE GANCIA DE VELOCIDAD

LEGv8 Pipeline

- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

PIPE LINE

F	D	E	M	W									
	F	D	E	M	W								
		F	D	E	M	W							
			F	D	E	M	W						
				F	D	E	M	W					
					F	D	E	M	W				
						F	D	E	M	W			
							F	D	E	M	W		
								F	D	E	M	W	
									F	D	E	M	W

Pipeline Performance

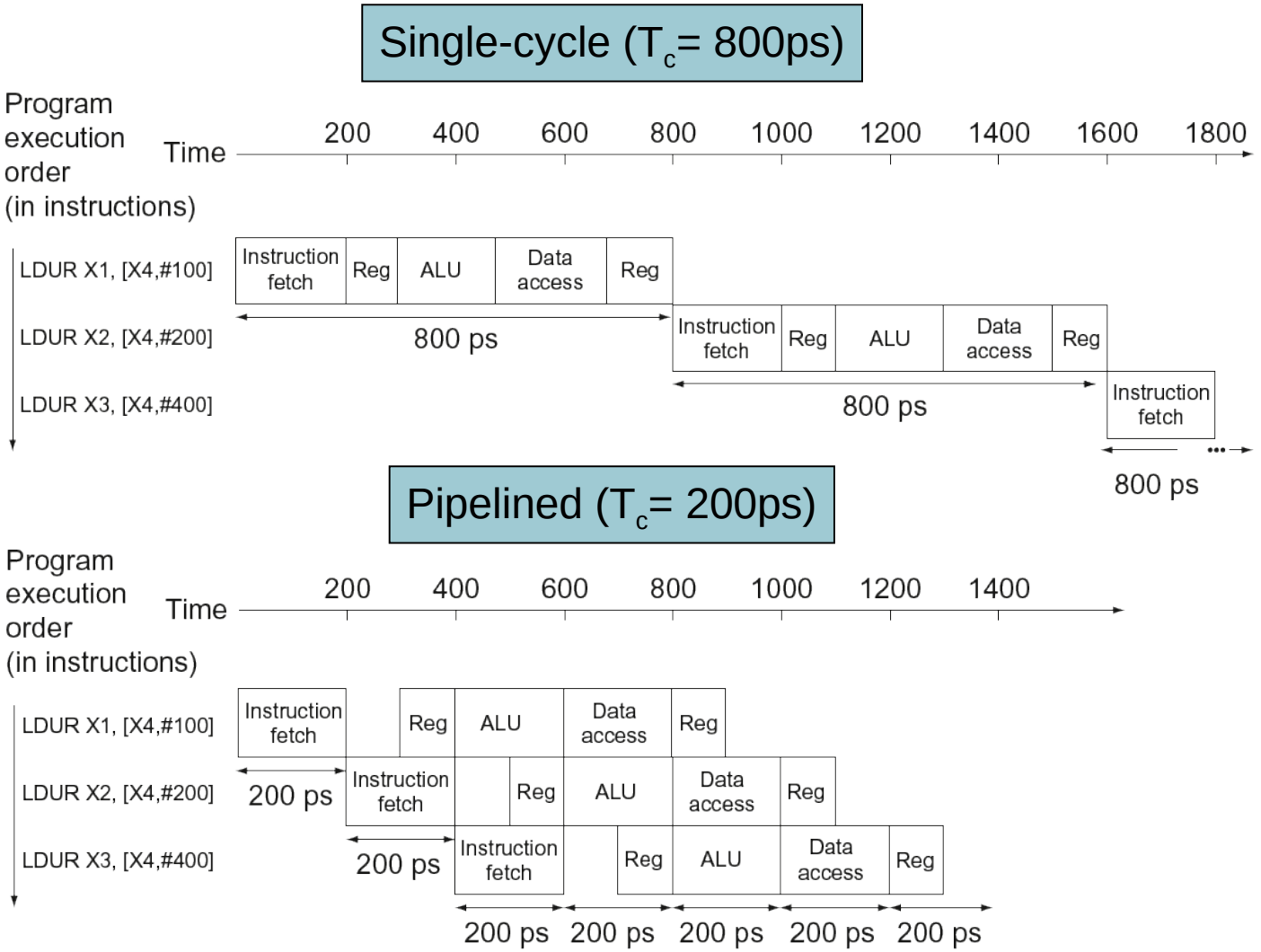
- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
LDUR	200ps	100 ps	200ps	200ps	100 ps	800ps
STUR	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
CBZ	200ps	100 ps	200ps			500ps

PIPE LINE

F	D	E	M	W									
	F	D	E	M	W								
		F	D	E	M	W							
			F	D	E	M	W						
				F	D	E	M	W					
					F	D	E	M	W				
						F	D	E	M	W			
							F	D	E	M	W		
								F	D	E	M	W	
									F	D	E	M	W

Pipeline Performance



Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time
  - Time between instructions$_{\text{pipelined}}$
    = $\dfrac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$
- If not balanced, speedup is less
- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

Pipelining and ISA Design

- LEGv8 ISA designed for pipelining
  - All instructions are 32-bits
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and regular instruction formats
    - Can decode and read registers in one step
  - Load/store addressing
    - Can calculate address in 3rd stage, access memory in 4th stage
  - Alignment of memory operands
    - Memory access takes only one cycle

Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

Structure Hazards

- Conflict for use of a resource
- In LEGv8 pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall* for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

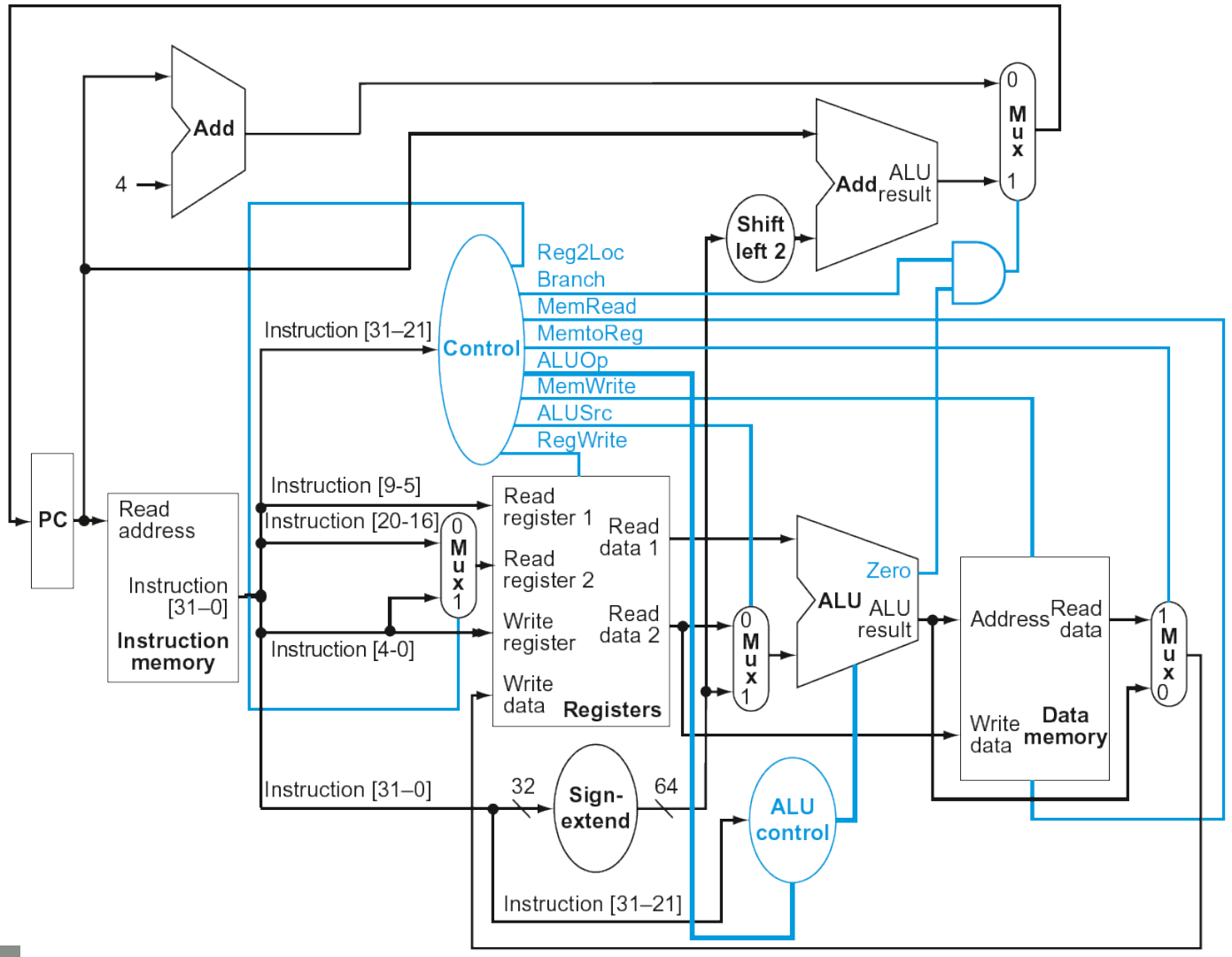
Structure Hazards

F	D	E	M	W									
	F	D	E	M	W								
		F	D	E	M	W							
			F	D	E	M	W						
				F	D	E	M	W					
					F	D	E	M	W				
						F	D	E	M	W			
							F	D	E	M	W		
								F	D	E	M	W	
									F	D	E	M	W

Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - `ADD X19, X0, X1`
    `SUB X2, X19, X3`

Datapath With Control



The Main Control Unit

- Control signals derived from instruction

Field	opcode	Rm	shamt	Rn	Rd
Bit positions	31:21	20:16	15:10	9:5	4:0

a. R-type instruction

Field	1986 or 1984	address	0	Rn	Rt
Bit positions	31:21	20:12	11:10	9:5	4:0

b. Load or store instruction

Field	180	address	Rt
Bit positions	31:26	23:5	4:0

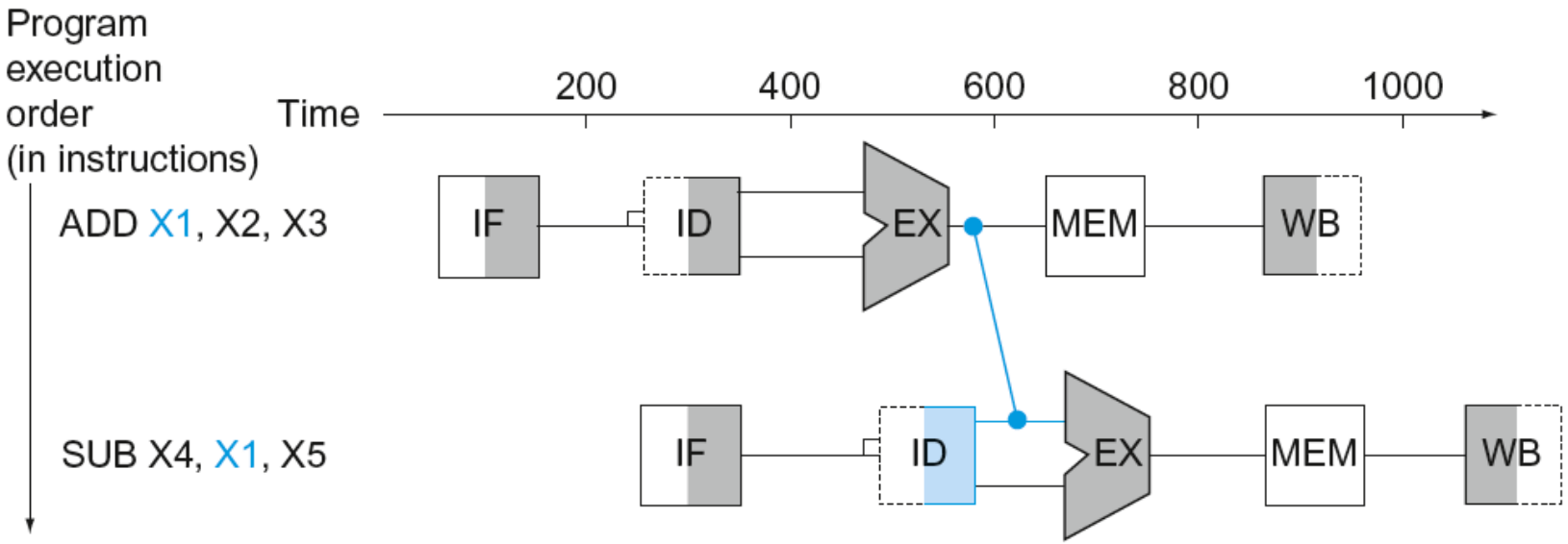
c. Conditional branch instruction

Data Hazards

		1	2	3	4	5	6	7	8
1	ADD X19, X20, X3	F	D	E	M	W			
2	SUB X2,X19,X3		F						
2	SUB X2,X19,X3			F					
2	SUB X2,X19,X3				F	D	E	M	W
...	...	...	...	...	...	...	...	...	...

Forwarding (aka Bypassing)

- Use result when it is computed
  - Don’t wait for it to be stored in a register
  - Requires extra connections in the datapath



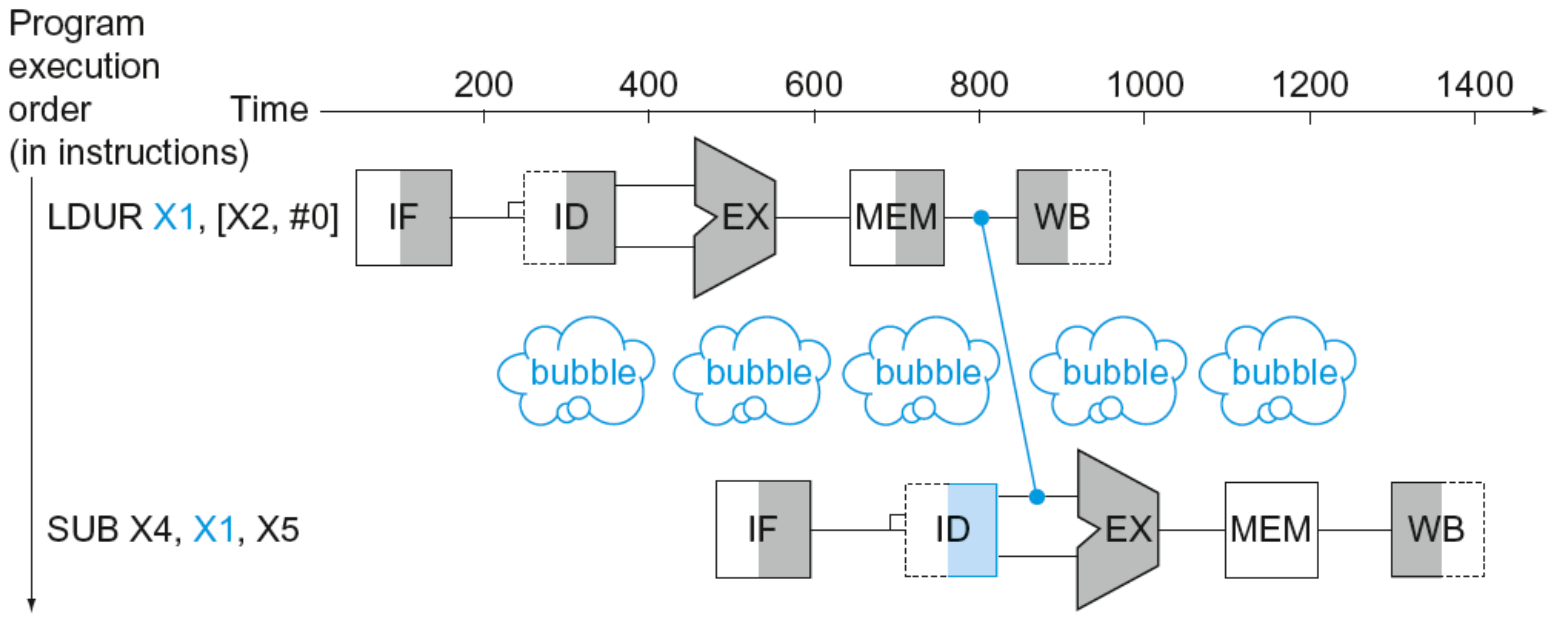
Forwarding (aka Bypassing)

- Use result when it is computed
  - Don’t wait for it to be stored in a register
  - Requires extra connections in the datapath

		1	2	3	4	5	6
1	ADD X19, X20, X3	F	D	E	M	W	
2	SUB X2,X19,X3		F	D	E	M	W
…	…	…	…	…	…	…	…

Load-Use Data Hazard

- Can’t always avoid stalls by forwarding
  - If value not computed when needed
  - Can’t forward backward in time!

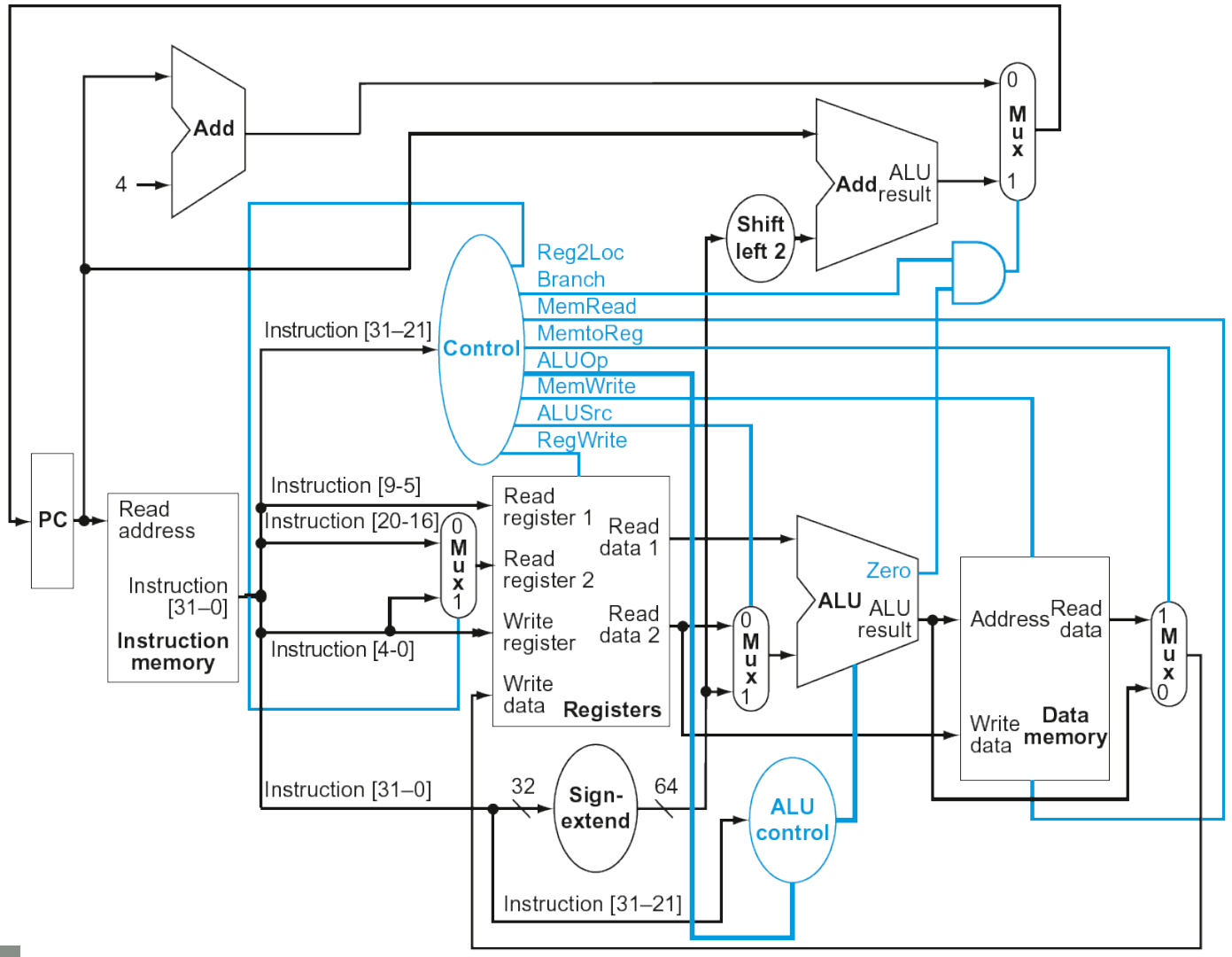


Load-Use Data Hazard

- Can’t always avoid stalls by forwarding
  - If value not computed when needed
  - Can’t forward backward in time!

		1	2	3	4	5	6	7
1	LDUR X1,[X2,#0]	F	D	E	M	W		
2	SUB X4,X1,X5		F					
2	SUB X4,X1,X5			F	D	E	M	W
…	…	…	…	…	…	…	…	…

Datapath With Control



The Main Control Unit

- Control signals derived from instruction

Field	opcode	Rm	shamt	Rn	Rd
Bit positions	31:21	20:16	15:10	9:5	4:0

a. R-type instruction

Field	1986 or 1984	address	0	Rn	Rt
Bit positions	31:21	20:12	11:10	9:5	4:0

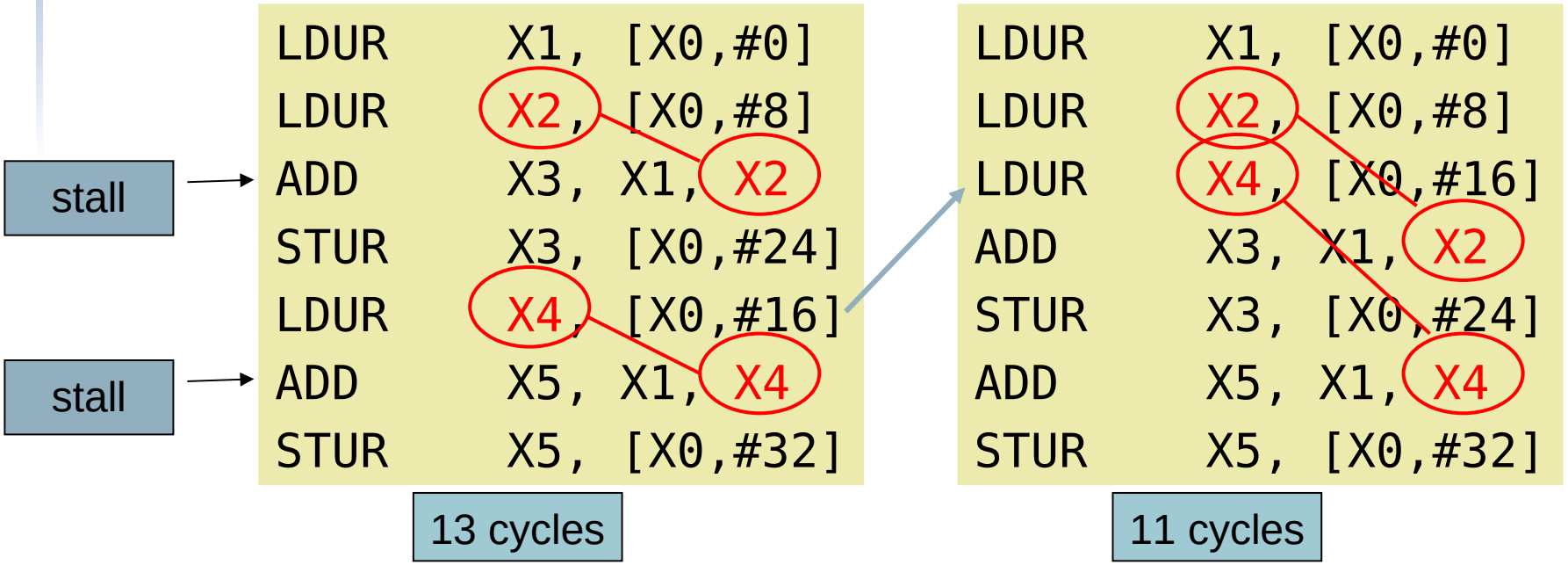
b. Load or store instruction

Field	180	address	Rt
Bit positions	31:26	23:5	4:0

c. Conditional branch instruction

Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for `A = B + E; C = B + F;`



Code Scheduling to Avoid Stalls

		1	2	3	4	5	6	7	8	9	10	11	12	13
1	LDUR X1,[X0,#0]	F	D	E	M	W								
2	LDUR X2,[X0,#8]		F	D	E	M	W							
3	ADD X3,X1,X2			F										
3	ADD X3,X1,X2				F	D	E	M	W					
4	STUR X3,[X0,#24]					F	D	E	M	W				
5	LDUR X4,[X0,#16]						F	D	E	M	W			
6	ADD X5,X1,X4							F						
6	ADD X5,X1,X4								F	D	E	M	W	
7	STUR X5,[X0,#32]									F	D	E	M	W
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

13 cycles

Code Scheduling to Avoid Stalls

		1	2	3	4	5	6	7	8	9	10	11
1	LDUR X1,[X0,#0]	F	D	E	M	W						
2	LDUR X2,[X0,#8]		F	D	E	M	W					
3	LDUR X4,[X0,#16]			F	D	E	M	W				
4	ADD X3,X1,X2				F	D	E	M	W			
5	STUR X3,[X0,#24]					F	D	E	M	W		
6	ADD X5,X1,X4						F	D	E	M	W	
7	STUR X5,[X0,#32]							F	D	E	M	W
...	...	...	...	...	...	...	...	...	...	...	...	...

11 cycles

Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can’t always fetch correct instruction
    - Still working on ID stage of branch
- In LEGv8 pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

Control Hazards

- Caso Extremo Esperar hasta el resultado
- (No ocurre en este diseño)

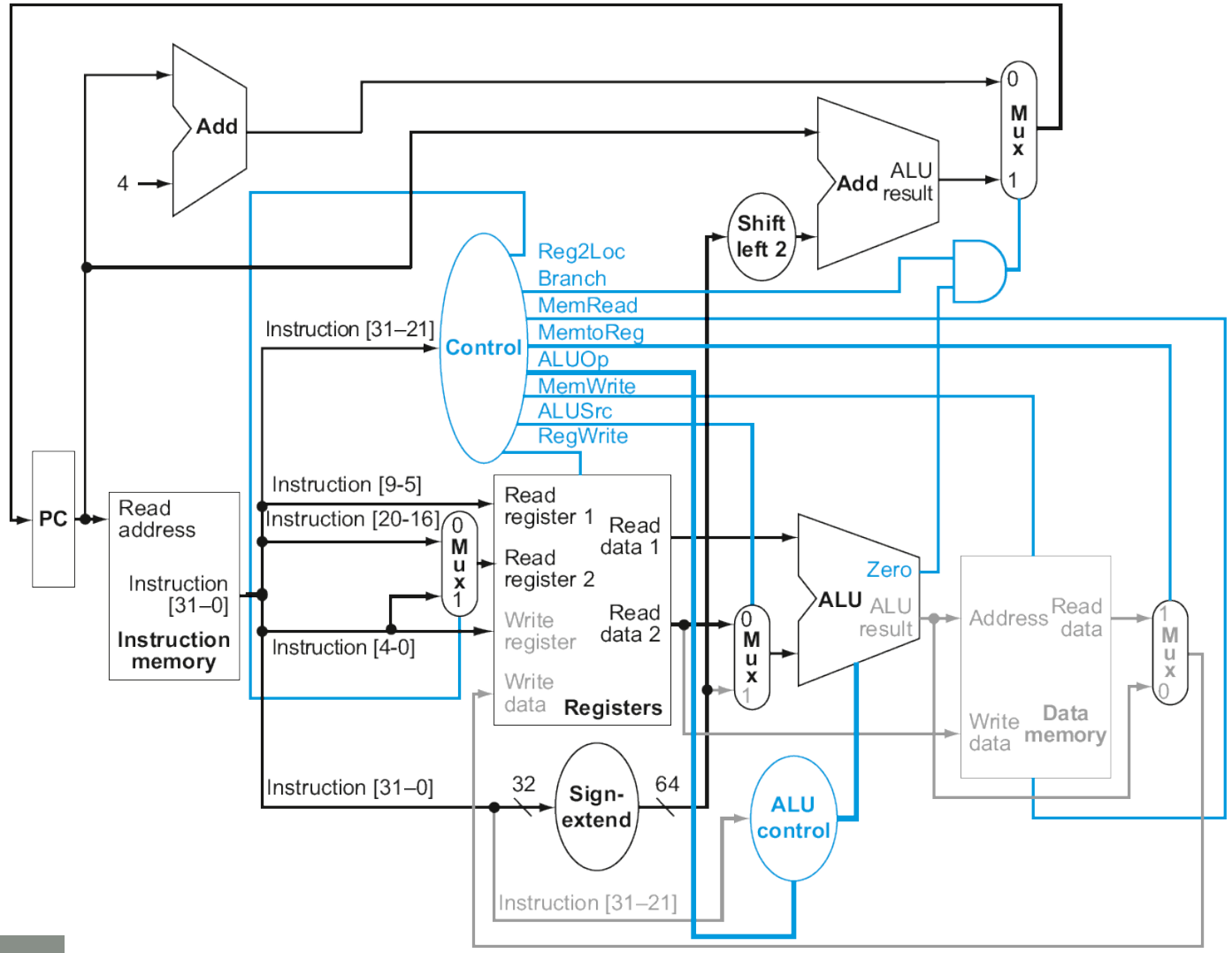
		1	2	3	4	5	6	7	8	9	10
1	ADD X3,X1,X2	F	D	E	M	W					
2	CBZ X1,40		F	D	E	M	W				
3	ORR X7,X8,X9						F	D	E	M	W
...	...	...	...	...	...	...	...	...	...	...	...

Control Hazards

- Situación «normal» en este diseño, sin mejoras

		1	2	3	4	5	6	7	8	9
1	ADD X3,X1,X2	F	D	E	M	W				
2	CBZ X1,40		F	D	E	M	W			
3	ORR X7,X8,X9					F	D	E	M	W
...	...	...	...	...	...	...	...	...	...	...

CBZ Instruction



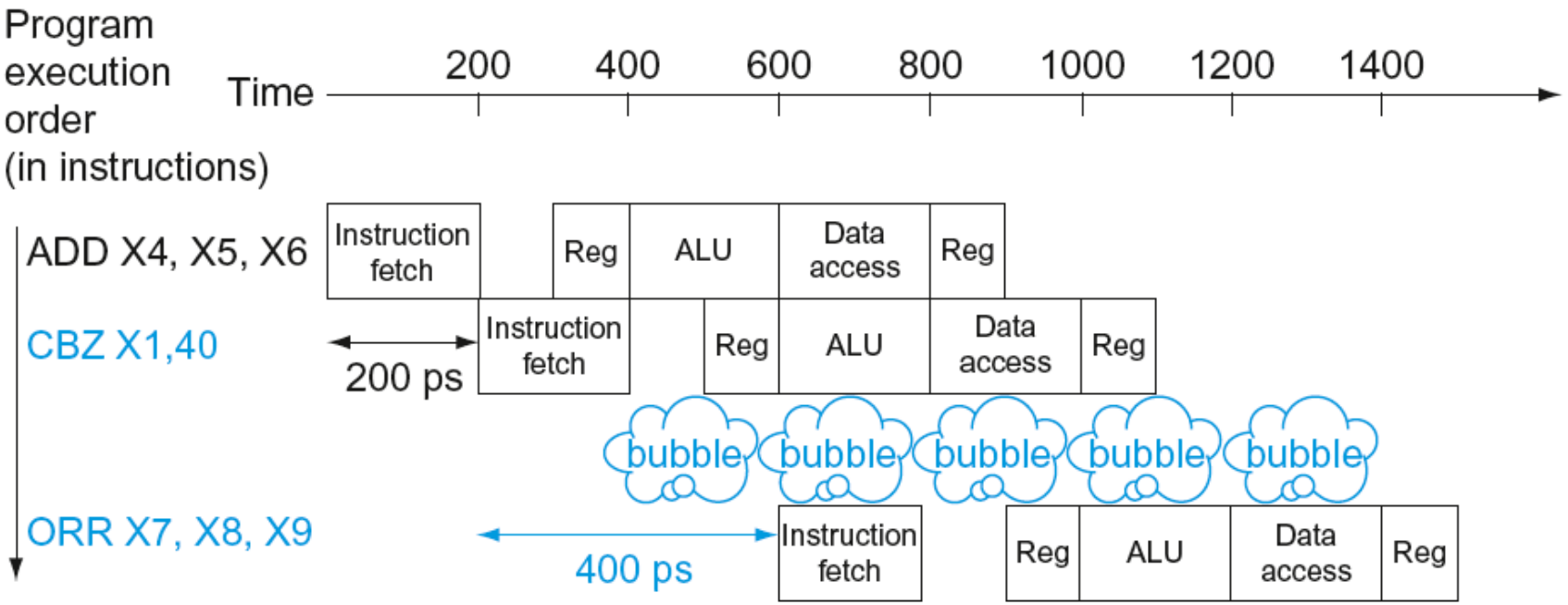
Control Hazards

- Mejora agregando hardware para resolver todo en la etapa D del salto

		1	2	3	4	5	6	7	8
1	ADD X3,X1,X2	F	D	E	M	W			
2	CBZ X1,40		F	D	E	M	W		
3	ORR X7,X8,X9				F	D	E	M	W
...	...	...	...	...	...	...	...	...	...

Stall on Branch

- Wait until branch outcome determined before fetching next instruction



Branch Prediction

- Longer pipelines can’t readily determine branch outcome early
  - Stall penalty becomes unacceptable
- Predict outcome of branch
  - Only stall if prediction is wrong
- In LEGv8 pipeline
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

Branch Prediction

- Si acertó…

		1	2	3	4	5	6	7
1	ADD X3,X1,X2	F	D	E	M	W		
2	CBZ X1,40		F	D	E	M	W	
3	ORR X7,X8,X9			F	D	E	M	W
…	…	…	…	…	…	…	…	…

Branch Prediction

- Si erró la predicción…

		1	2	3	4	5	6	7	8
1	ADD X3,X1,X2	F	D	E	M	W			
2	CBZ X1,40		F	D	E	M	W		
3	ORR X7,X8,X9			F					
3	XXXXXXXXXX				F	D	E	M	W
…	…	…	…	…	…	…	…	…	…

More-Realistic Branch Prediction

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken
- Dynamic branch prediction
  - Hardware measures actual branch behavior
    - e.g., record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

Pipeline Summary

The BIG Picture

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

Pipeline Primeros Ejercicios

- **Ejercicio 1:**
- En un microprocesador con tres etapas de pipeline: S_1 -> S_2 -> S_3, con tiempos de ejecución $T_1 = T_3 = T$ y $T_2 = 3T$.
- a) ¿Cuál de los tres segmentos o etapas causa la congestión? (el cuello de botella).
- b) Asumiendo que el segmento problemático se puede dividir en dos etapas consecutivas, ninguna de ellas con duración menor que T, ¿cuál sería la mejor partición posible? ¿Cuál sería el período de *clock* resultante para este nuevo pipeline de 4 etapas?
- c) Asumiendo que el segmento problemático se puede dividir en varias etapas consecutivas de duración T, ¿cuál es el período de *clock* del pipeline? ¿Cuál es el tiempo de ejecución entre instrucciones?

Pipeline Primeros Ejercicios

- **Ejercicio 2:**
- Asumiendo que las etapas de un procesador ARM tienen las siguientes latencias :

IF	ID	EX	MEM	WB
30ps	9ps	20ps	25ps	10ps

- a) ¿Cuánto tiempo se requiere en un microprocesador sin pipeline para completar la ejecución de una instrucción tipo R?
- b) ¿Cuánto tiempo se requiere en un microprocesador sin pipeline para completar la ejecución de la instrucción de mayor latencia, es decir, la latencia del procesador completo?
- c) Si se requiere ejecutar esa instrucción en un microprocesador con pipeline, ¿a qué velocidad debería trabajar el *clock*?

Pipeline Primeros Ejercicios

- d) ¿Cuál es el tiempo de ejecución de una instrucción en un microprocesador con pipeline? ¿Cada cuanto se ejecuta una nueva instrucción en este procesador?

- e) Si un microprocesador con pipeline ejecuta 3 instrucciones consecutivas ¿cuál es la ganancia de velocidad de un procesador con pipeline respecto de uno sin pipeline? ¿Y si se ejecutan 1000 instrucciones consecutivas?

$$Ganancia\ de\ velocidad = \frac{Tiempo\ de\ ejecución\ sin\ pipline}{Tiempo\ de\ ejecución\ con\ pipline}$$

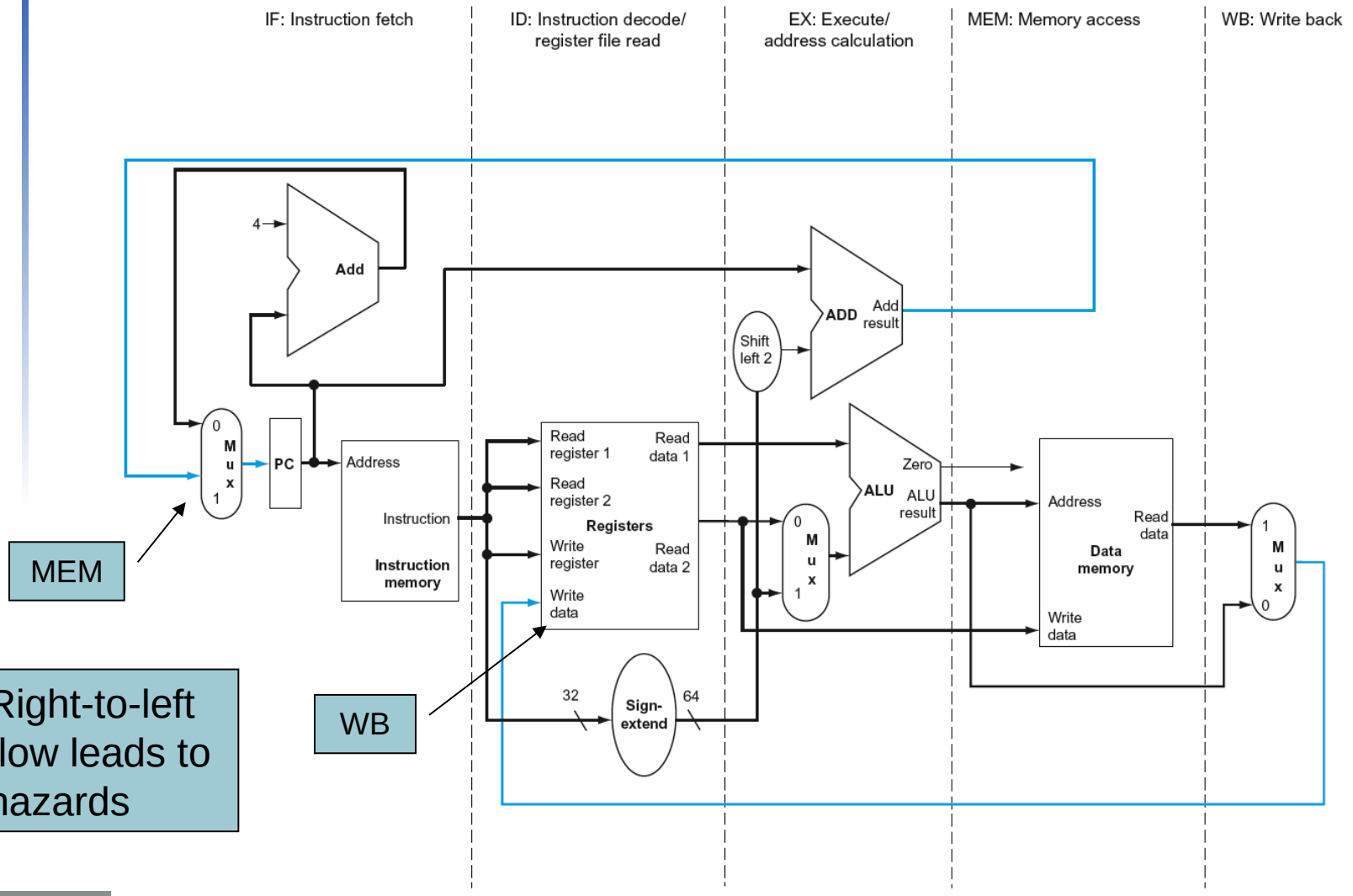
Pipeline Primeros Ejercicios

- **Ejercicio 3:**
- Asumiendo que las etapas individuales del pipeline de dos procesadores ARM distintos tienen las siguientes latencias:

Caso	IF	ID	EX	MEM	WB	Unidad
1	300	400	350	500	100	ps
2	200	150	120	190	140	ps

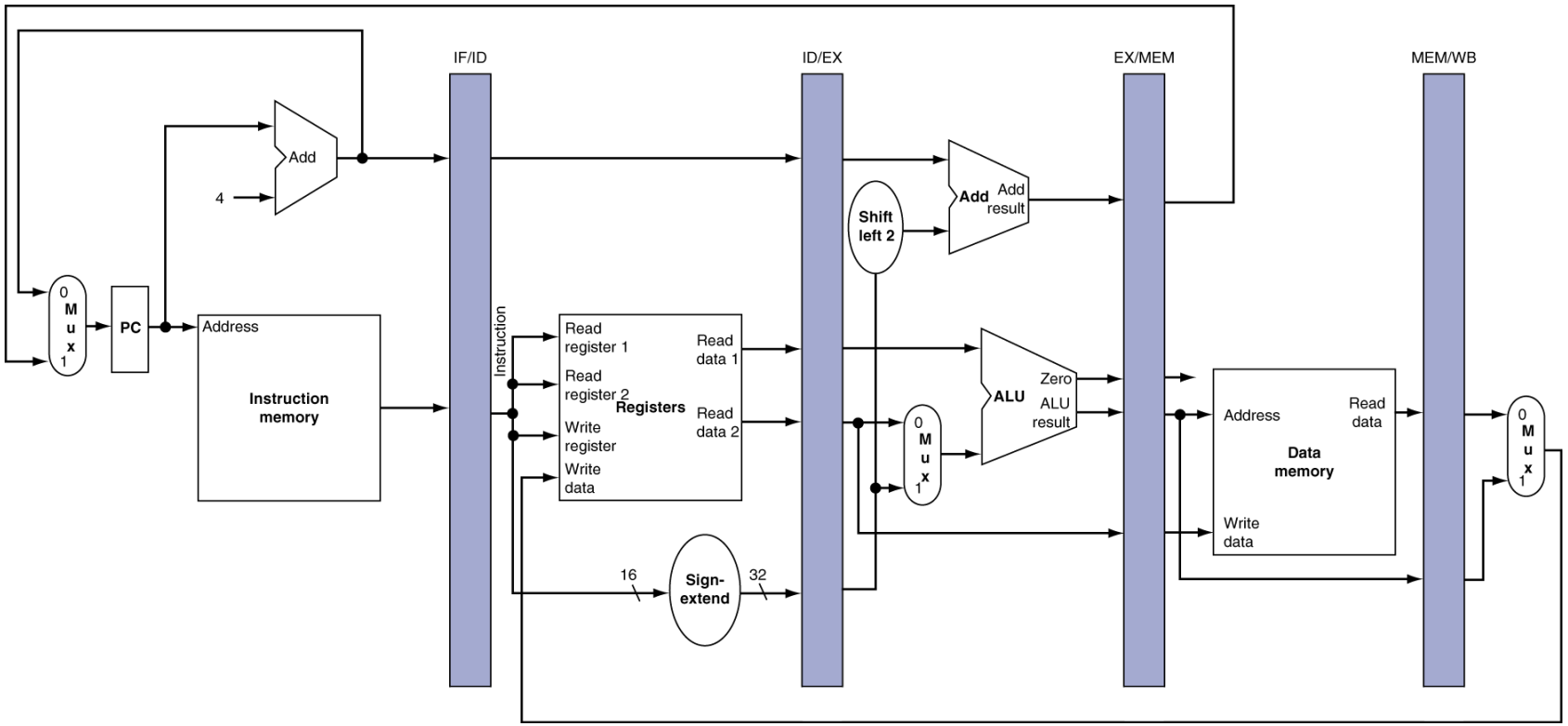
- a) ¿Cuál es el ciclo de *clock* para la versión con y sin pipeline para cada caso?
- b) ¿Cuál es la latencia de la instrucción **LDUR** para ambos, considerando las versiones de procesador con y sin pipeline?
- c) Si se pudiera partir una etapa del pipeline en dos nuevas etapas, cada una con la mitad de la latencia de la etapa original, ¿Que etapa elegiría y cuál será el nuevo ciclo de *clock*?

LEGv8 Pipelined Datapath



Pipeline registers

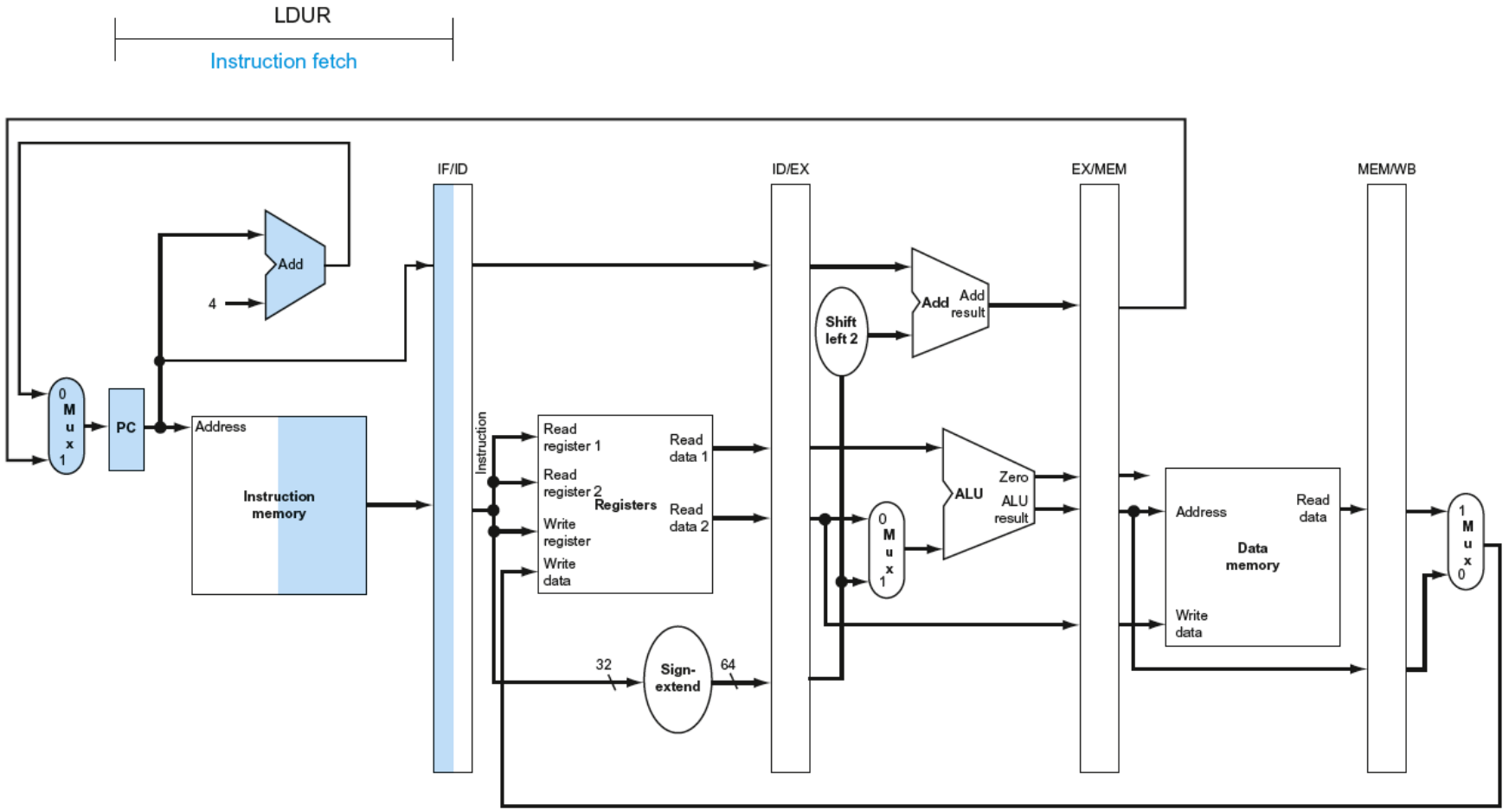
- Need registers between stages
  - To hold information produced in previous cycle



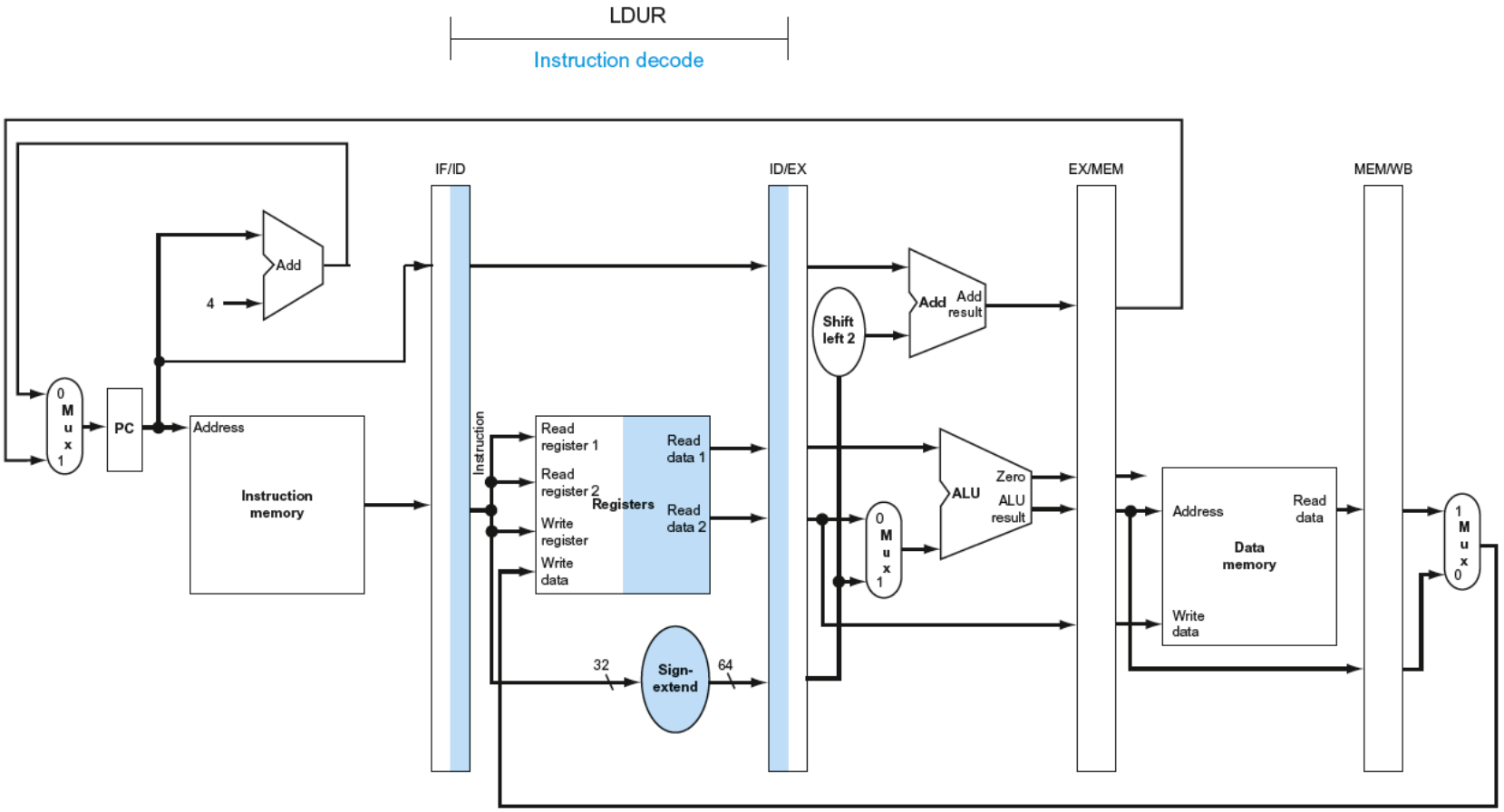
Pipeline Operation

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - “Single-clock-cycle” pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. “multi-clock-cycle” diagram
    - Graph of operation over time
- We’ll look at “single-clock-cycle” diagrams for load & store

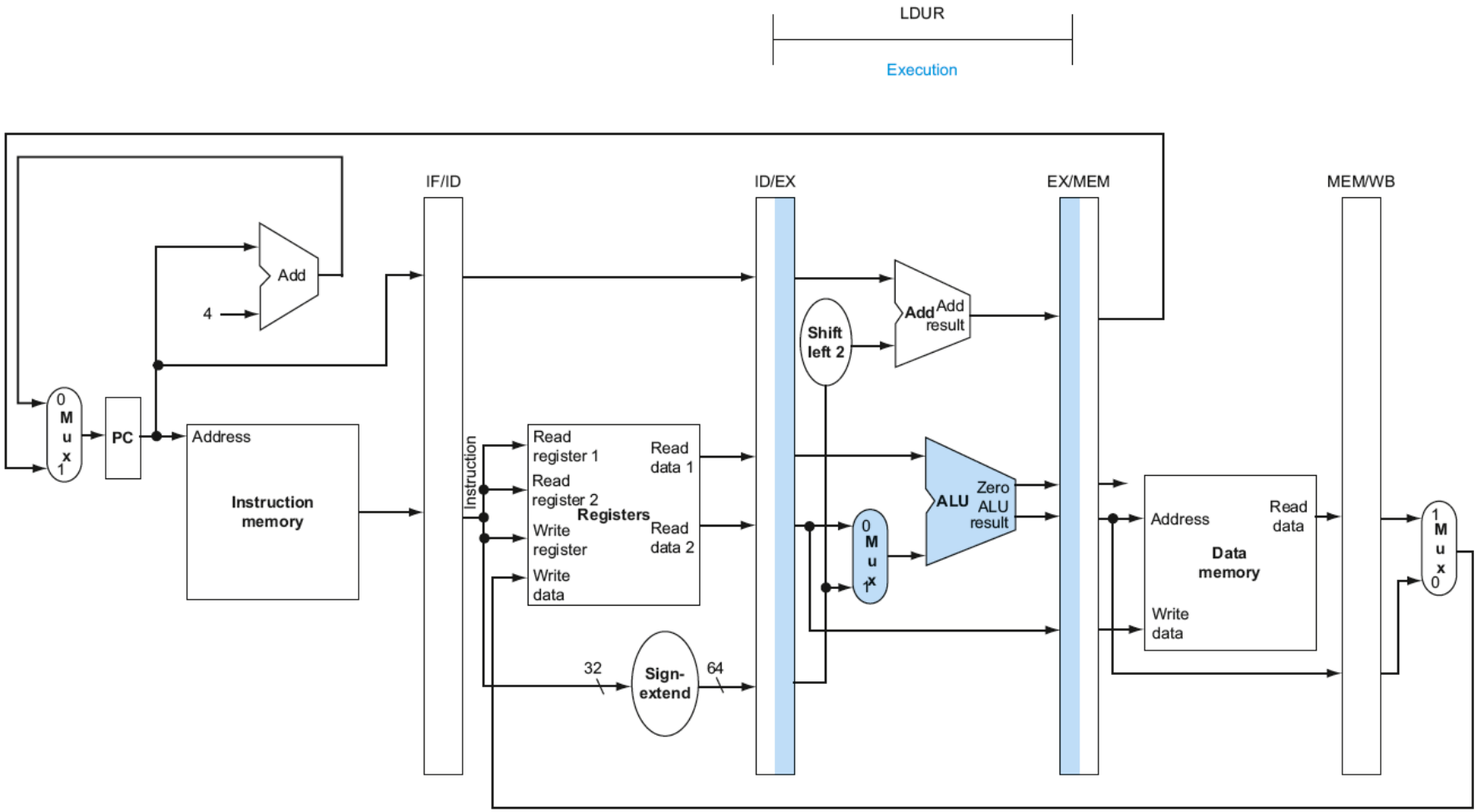
IF for Load, Store, …



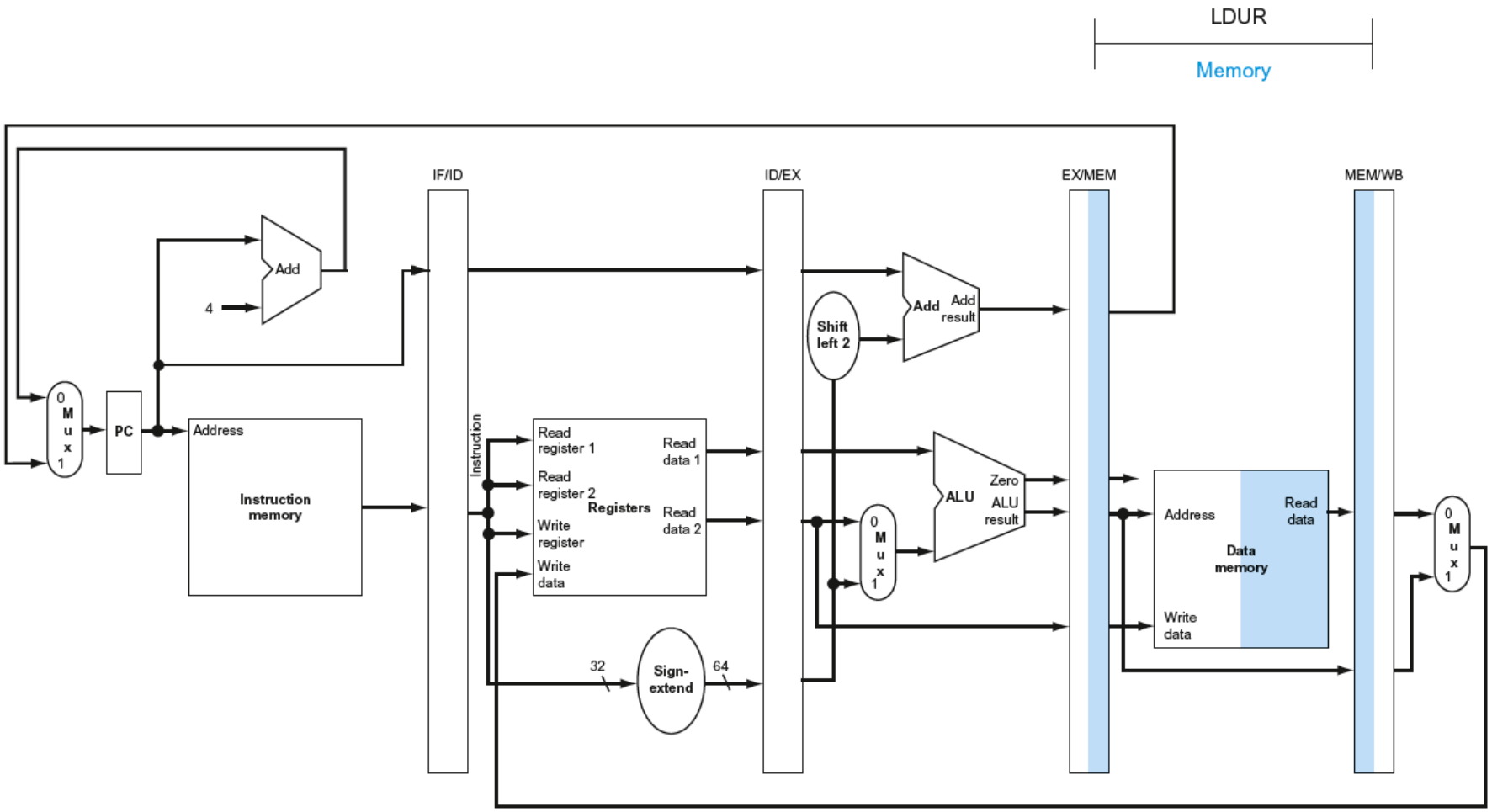
ID for Load, Store, …



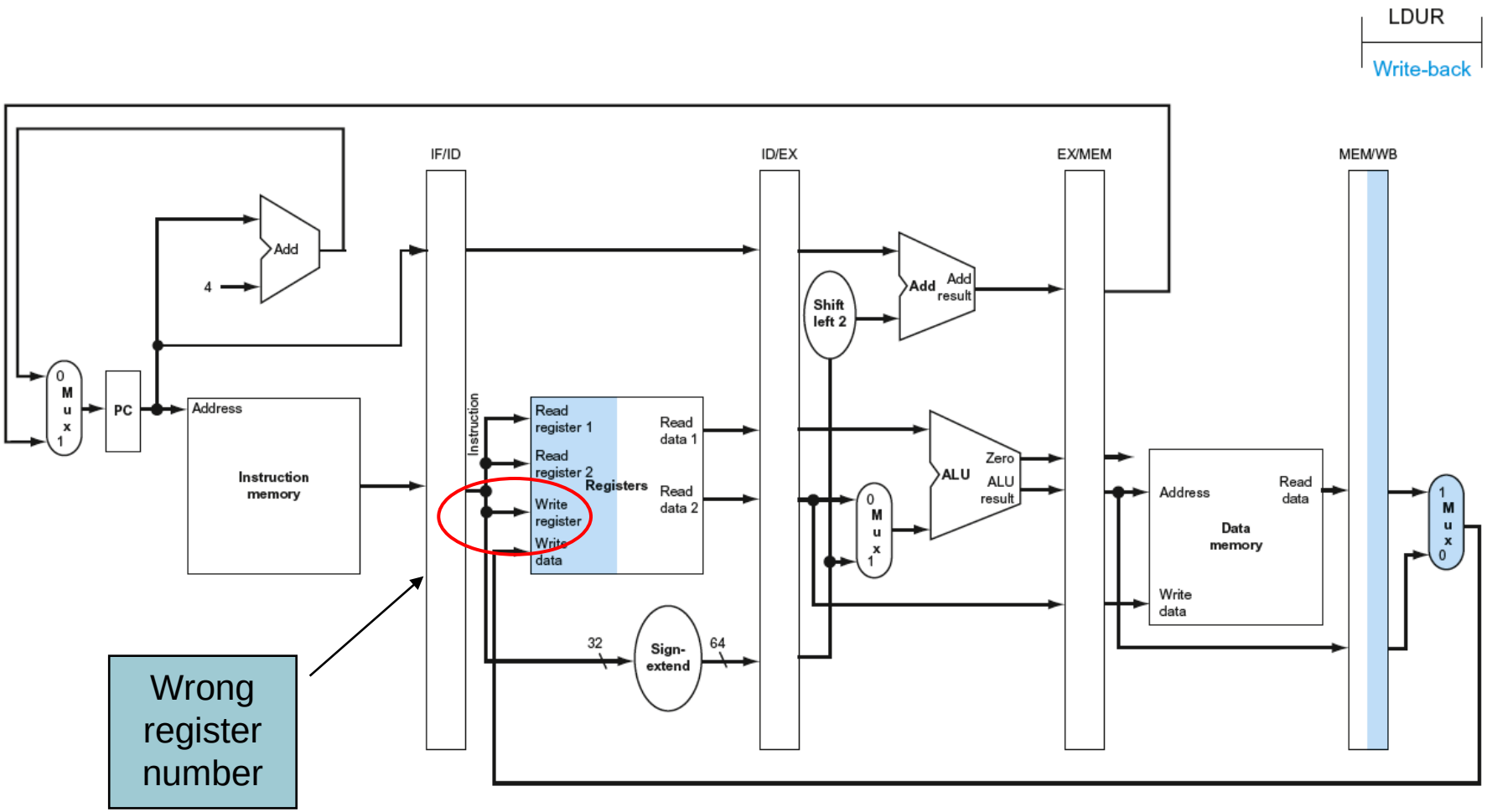
EX for Load


LDUR
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64

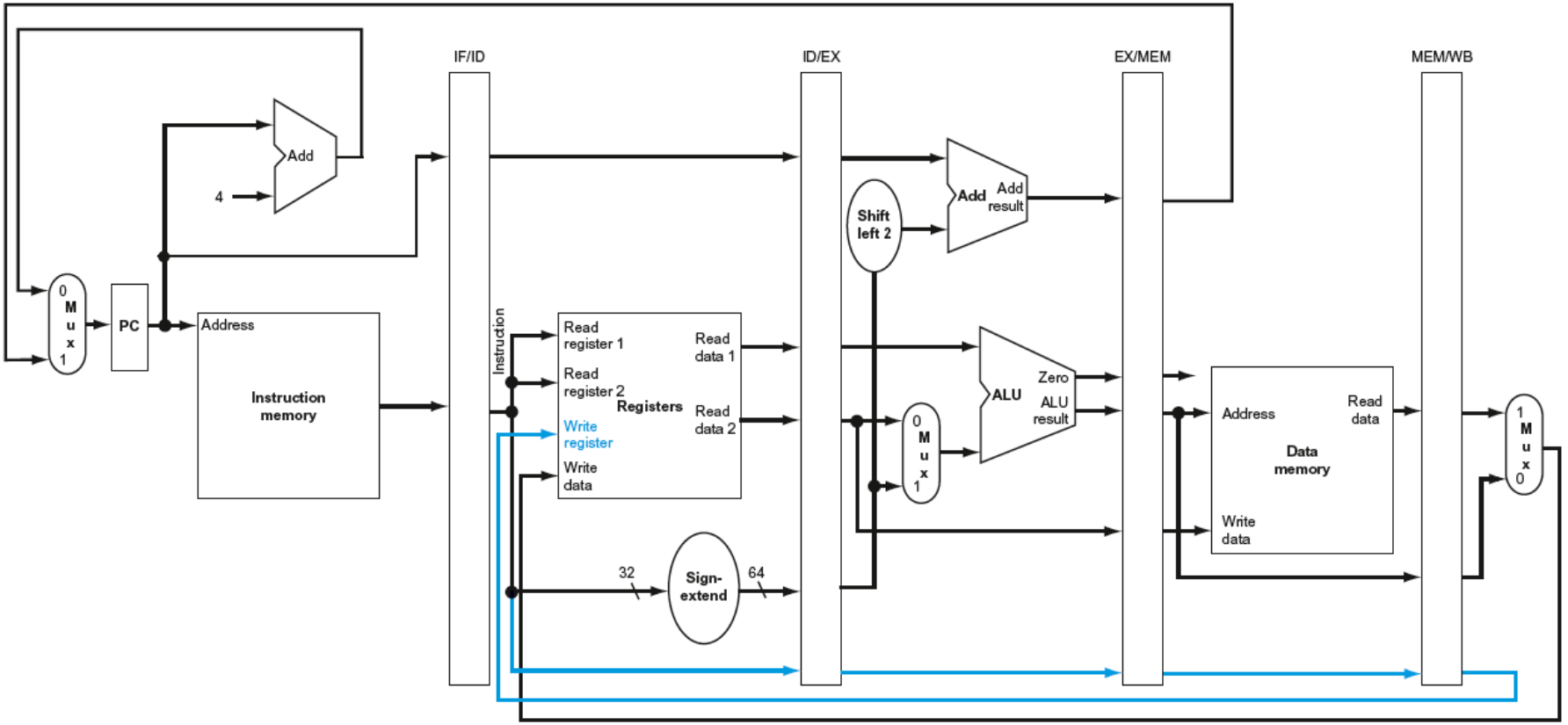
MEM for Load


LDUR
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
32
Sign-extend
64

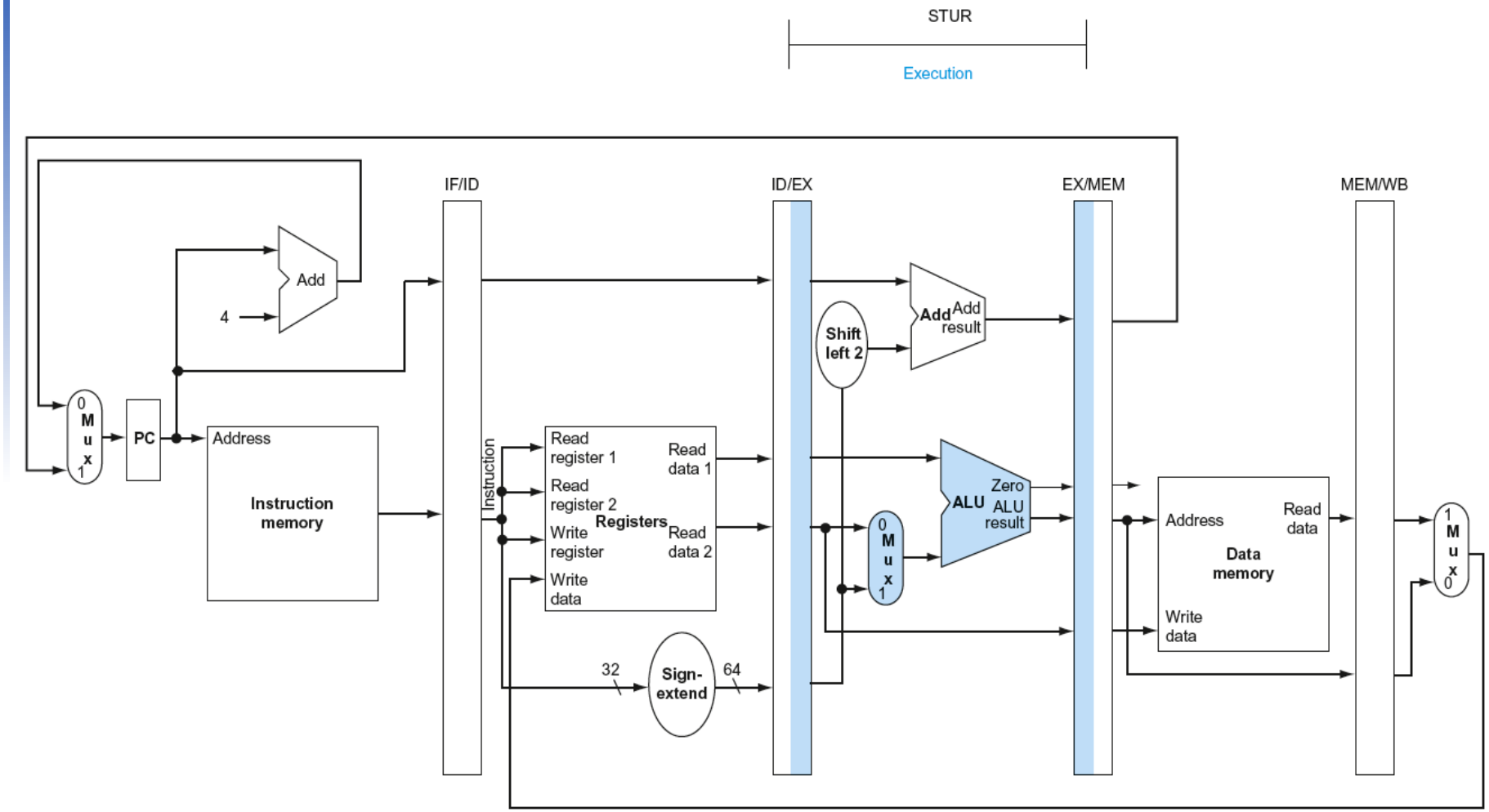
WB for Load


LDUR
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64
Wrong register number

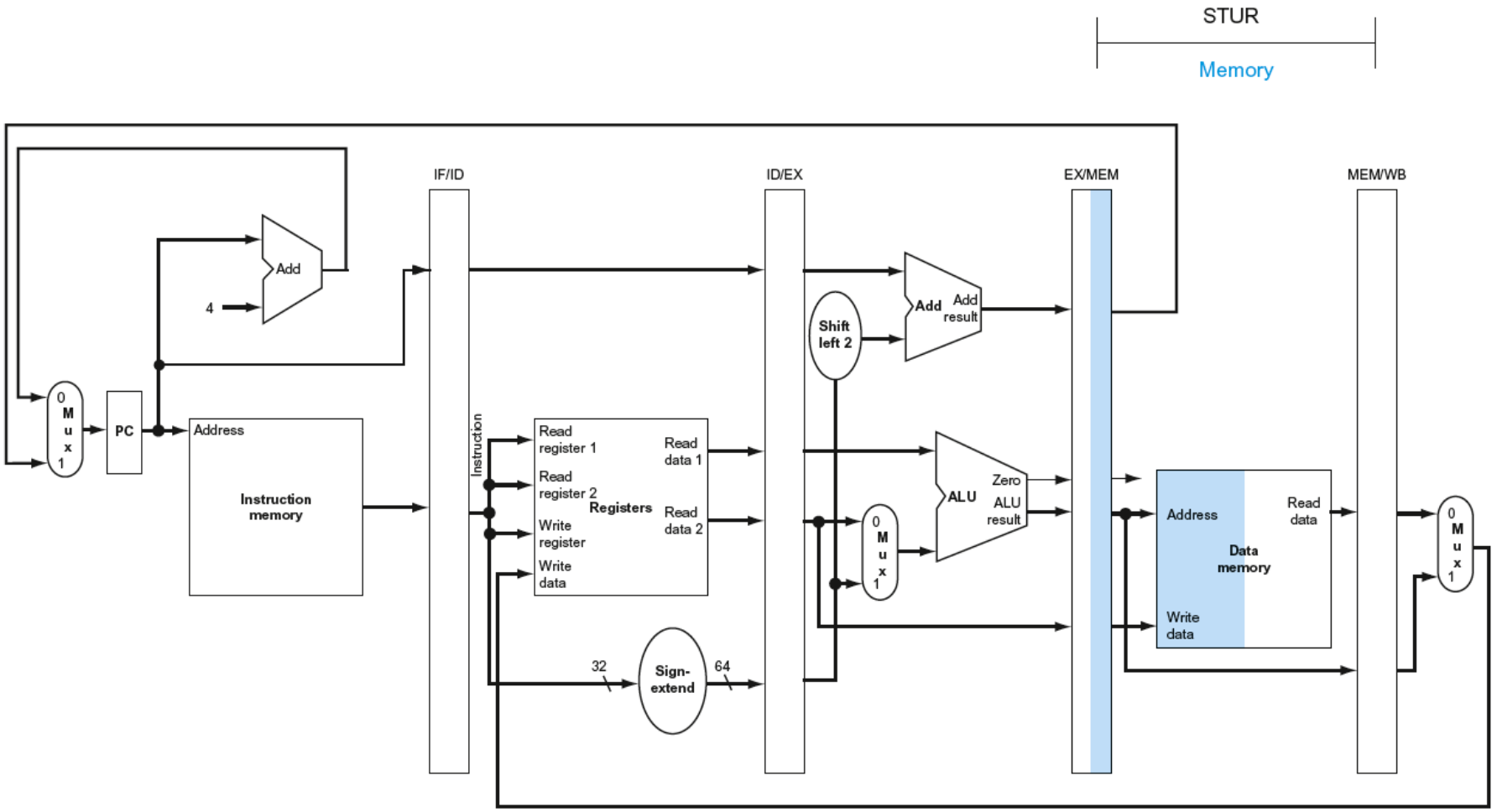
Corrected Datapath for Load



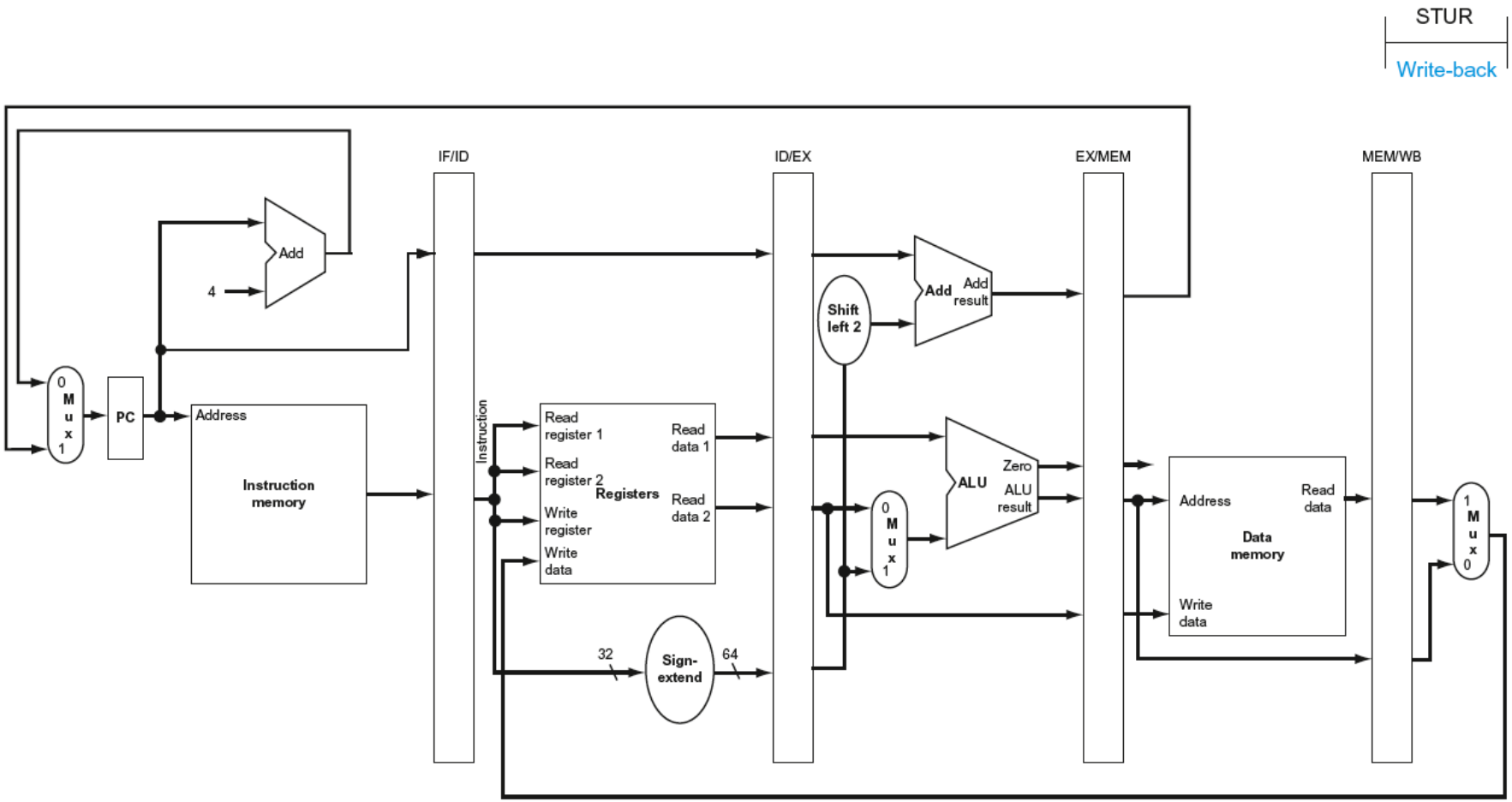
EX for Store


STUR
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
32
Sign-extend
64
Address
Read data
Data memory
Write data

MEM for Store

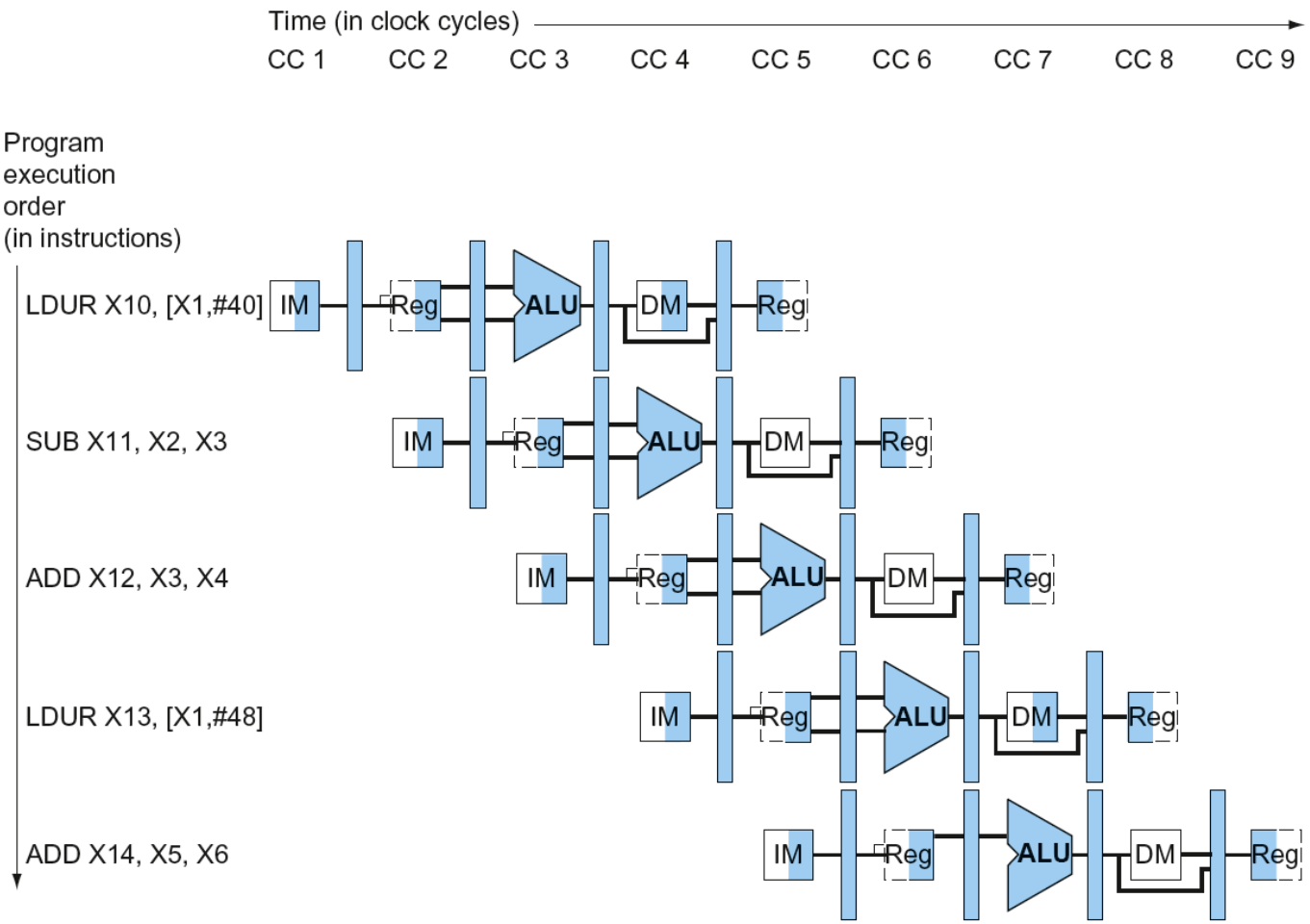


WB for Store


STUR
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
32
Sign-extend
64
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data

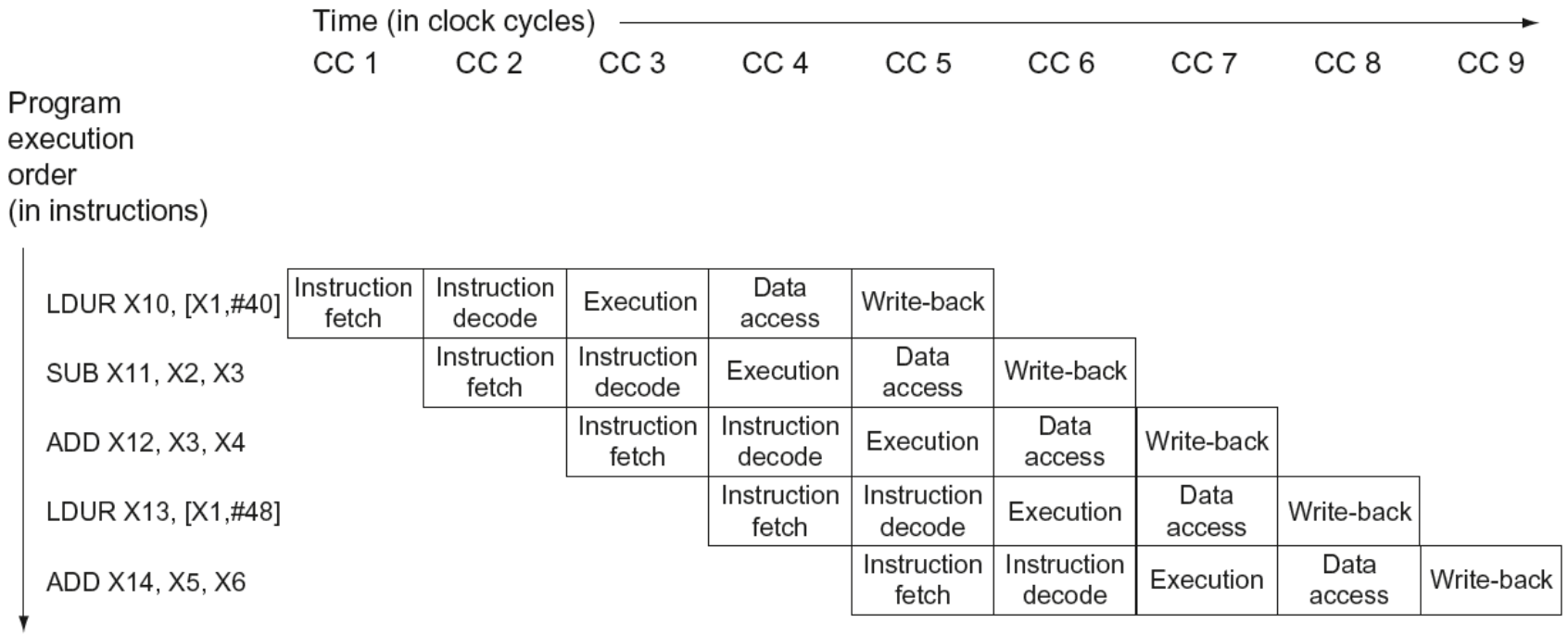
Multi-Cycle Pipeline Diagram

- Form showing resource usage



Multi-Cycle Pipeline Diagram

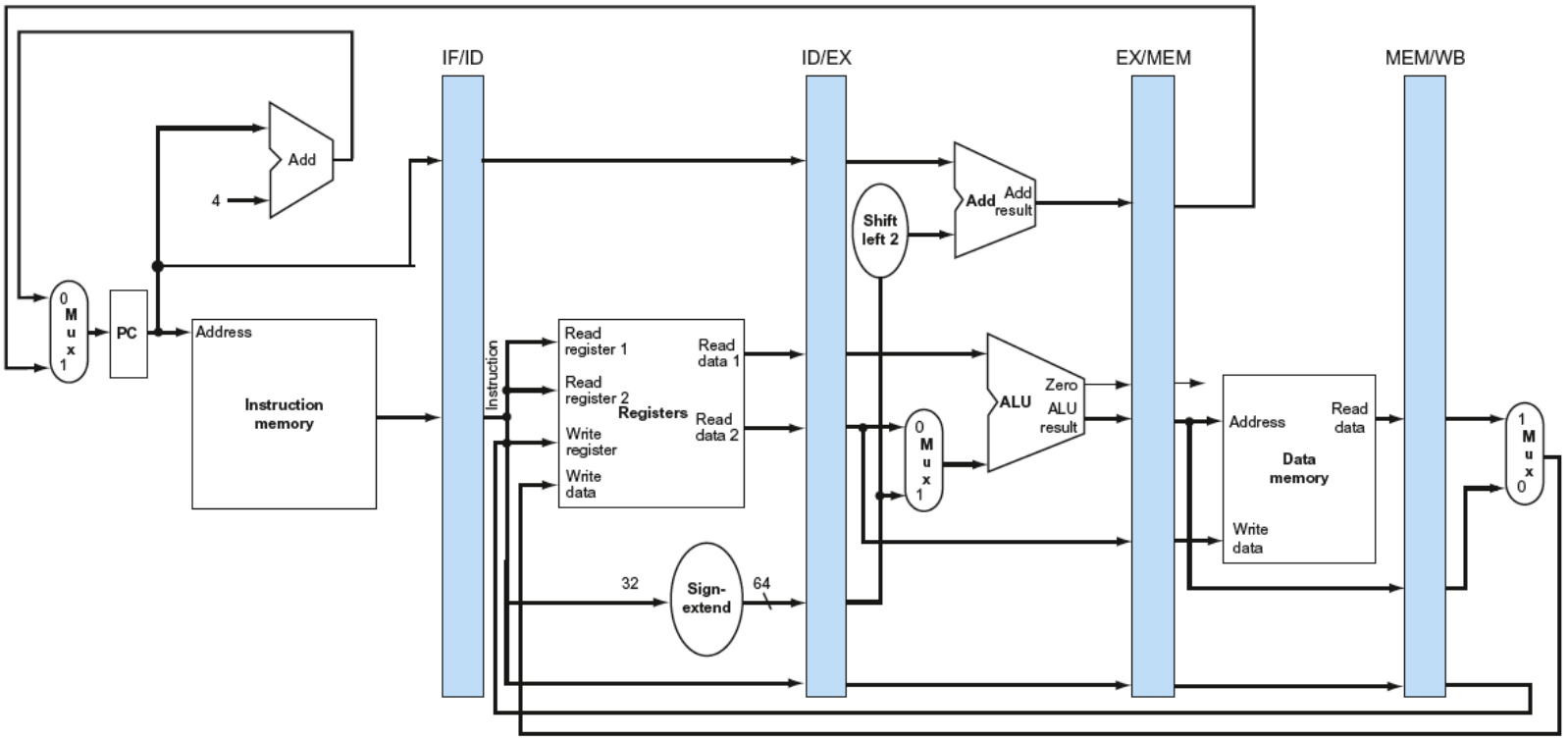
- Traditional form



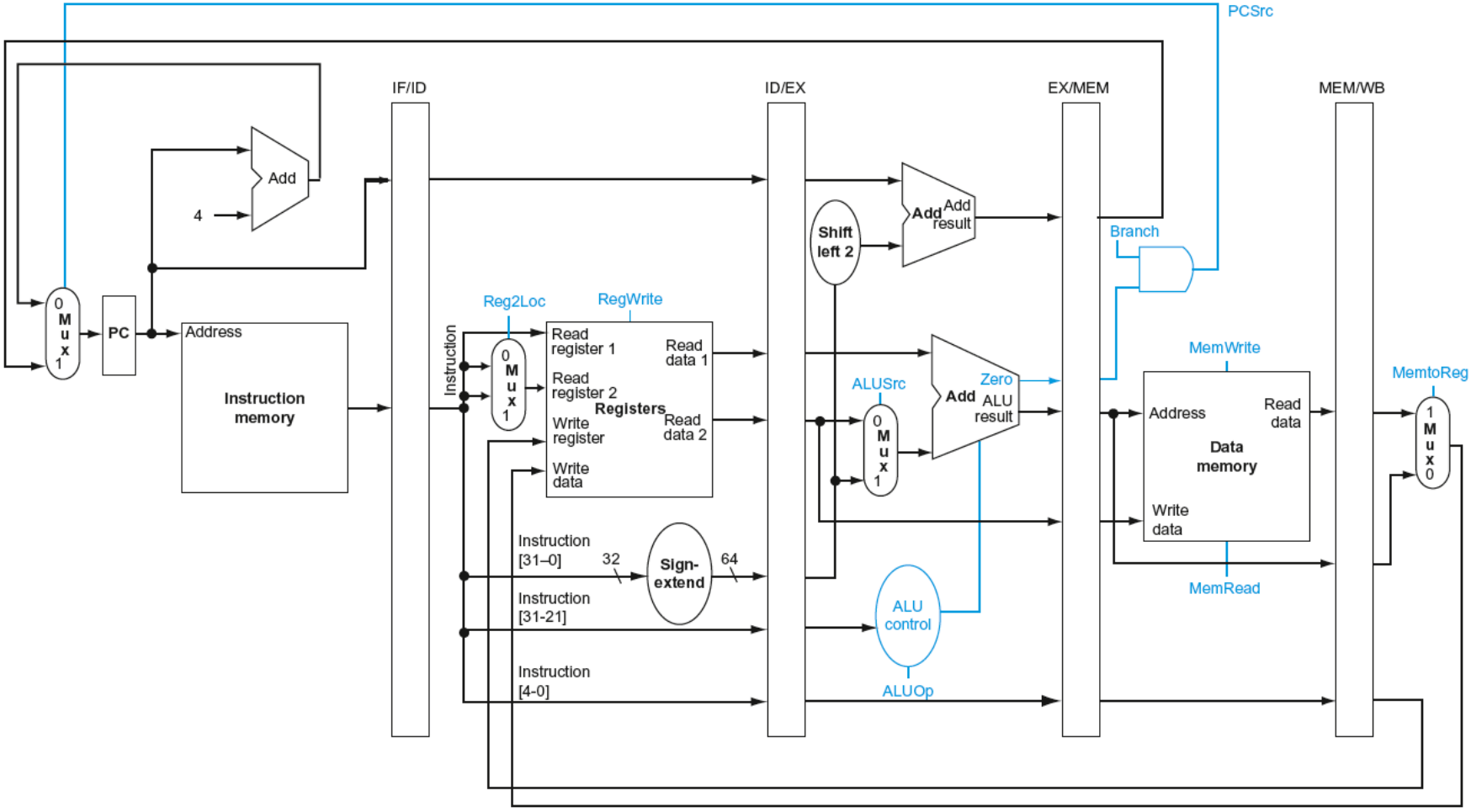
Single-Cycle Pipeline Diagram

- State of pipeline in a given cycle

ADD X14, X5, X6	LDUR X13, [X1,48]	ADD X12, X3, X4	SUB X11, X2, X3	LDUR X10, [X1,40]
Instruction fetch	Instruction decode	Execution	Memory	Write-back

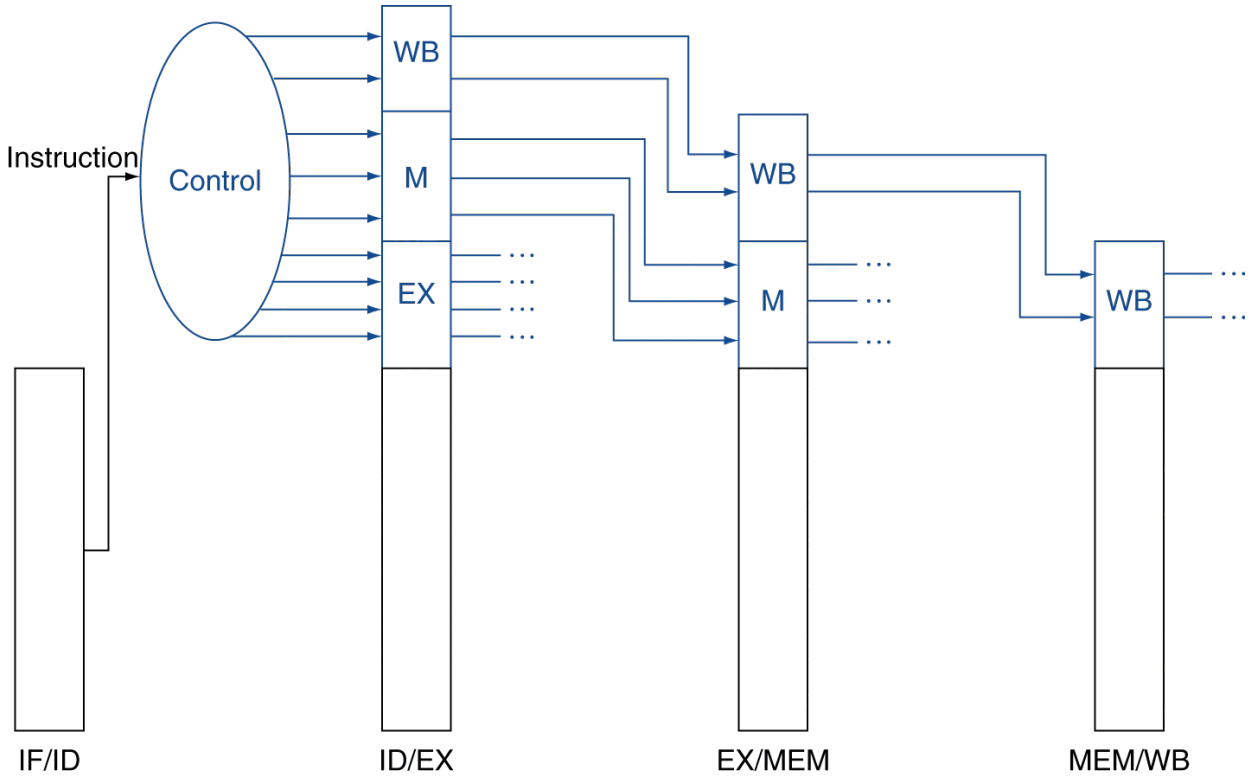


Pipelined Control (Simplified)

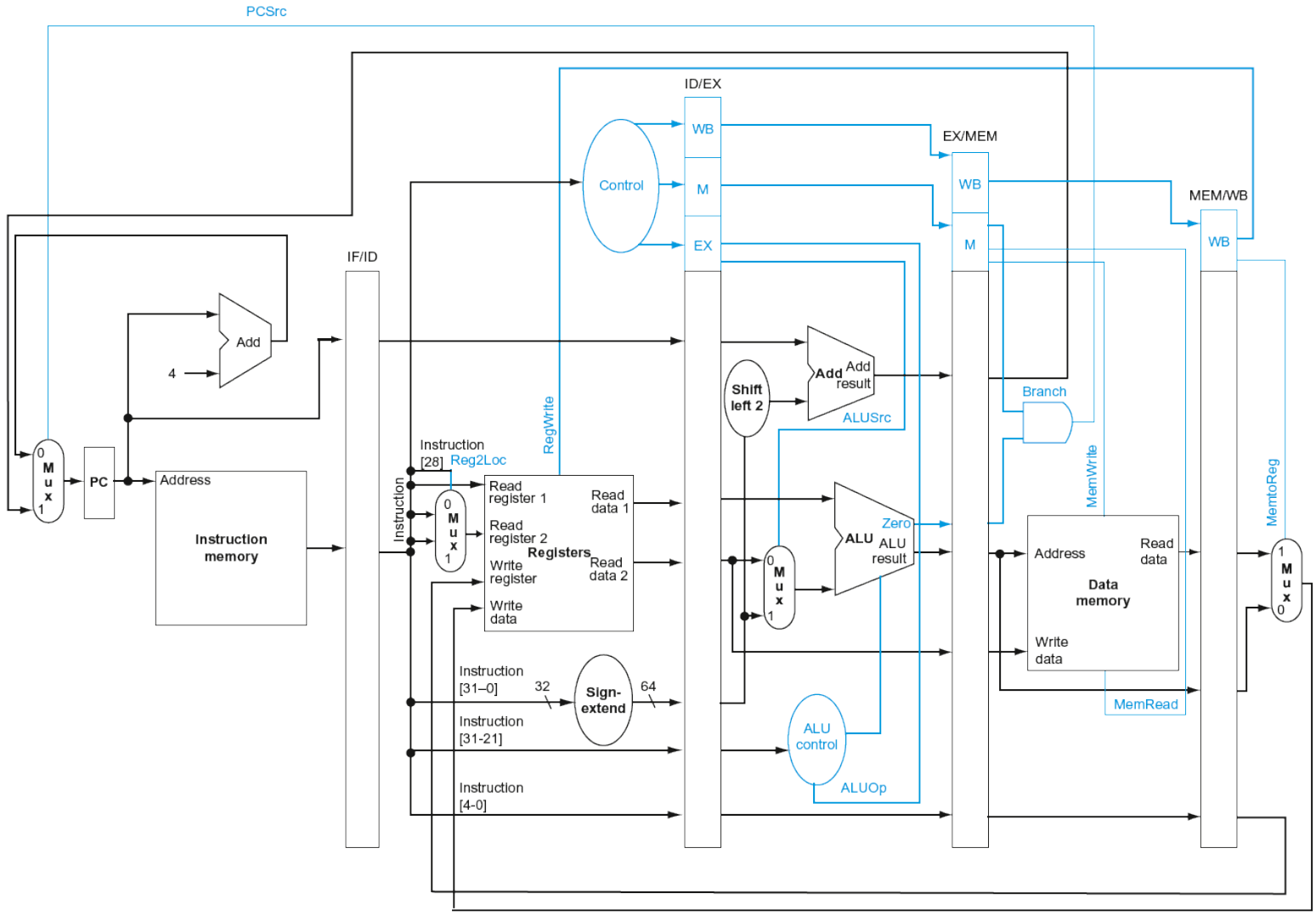


Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation



Pipelined Control



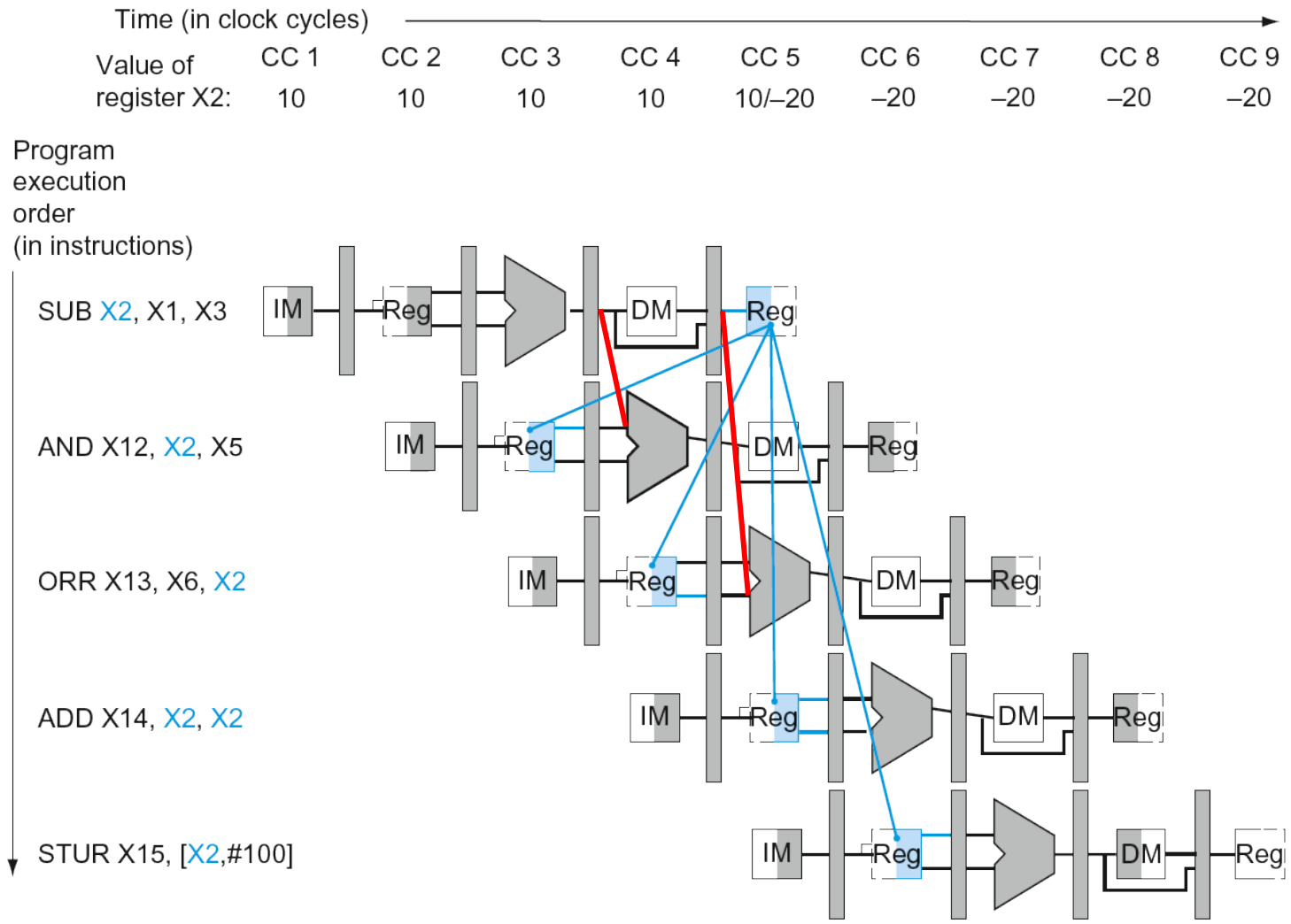
Data Hazards in ALU Instructions

- Consider this sequence:

```
SUB  X2, X1,X3
AND  X12,X2,X5
OR   X13,X6,X2
ADD  X14,X2,X2
STUR X15,[X2,#100]
```

- We can resolve hazards with forwarding
  - How do we detect when to forward?

Dependencies & Forwarding



Detecting the Need to Forward

- Pass register numbers along pipeline
  - e.g., ID/EX.RegisterRs = register number for Rs sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRn1, ID/EX.RegisterRm2
- Data hazards when
  1a. EX/MEM.RegisterRd = ID/EX.RegisterRn1
  1b. EX/MEM.RegisterRd = ID/EX.RegisterRm2
  2a. MEM/WB.RegisterRd = ID/EX.RegisterRn1
  2b. MEM/WB.RegisterRd = ID/EX.RegisterRm2

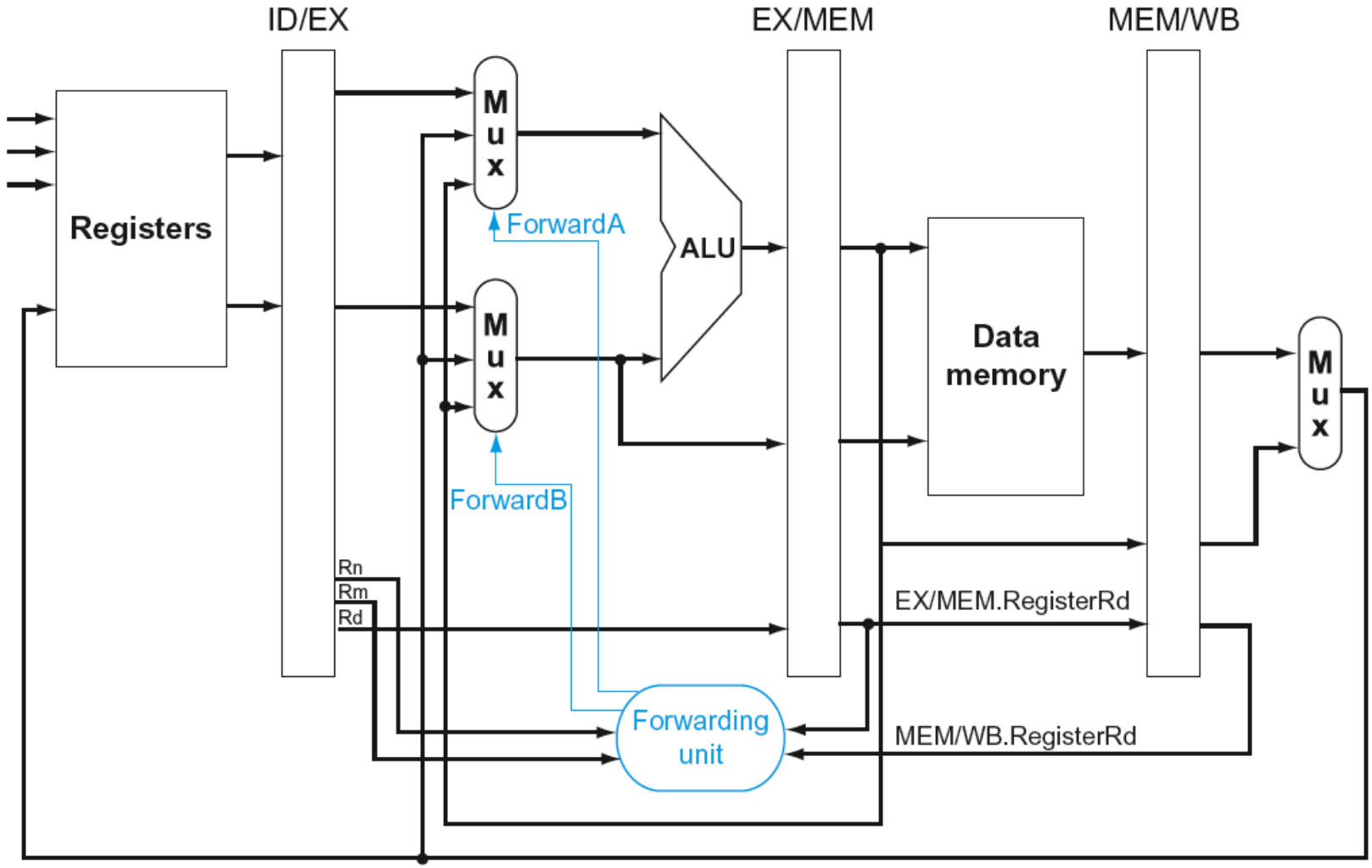
Fwd from EX/MEM pipeline reg

Fwd from MEM/WB pipeline reg

Detecting the Need to Forward

- But only if forwarding instruction will write to a register!
  - EX/MEM.RegWrite, MEM/WB.RegWrite
- And only if Rd for that instruction is not XZR
  - EX/MEM.RegisterRd ≠ 31, MEM/WB.RegisterRd ≠ 31

Forwarding Paths



Forwarding Conditions

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Double Data Hazard

- Consider the sequence:

```
add X1,X1,X2
add X1,X1,X3
add X1,X1,X4
```

- Both hazards occur
  - Want to use the most recent
- Revise MEM hazard condition
  - Only fwd if EX hazard condition isn't true

Double Data Hazard

				1	2	3	4	5	6	7
1	ADD	X1,X1,X2		F	D	E	M	W		
2	ADD	X1,X1,X3			F	D	E	M	W	
3	ADD	X1,X1,X4				F	D	E	M	W

Revised Forwarding Condition

- MEM hazard
  - if (MEM/WB.RegWrite

    and (MEM/WB.RegisterRd ≠ 31)

    and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 31)

    and (EX/MEM.RegisterRd ≠ ID/EX.RegisterRn1))

    and (MEM/WB.RegisterRd = ID/EX.RegisterRn1)) ForwardA = 01
  - if (MEM/WB.RegWrite

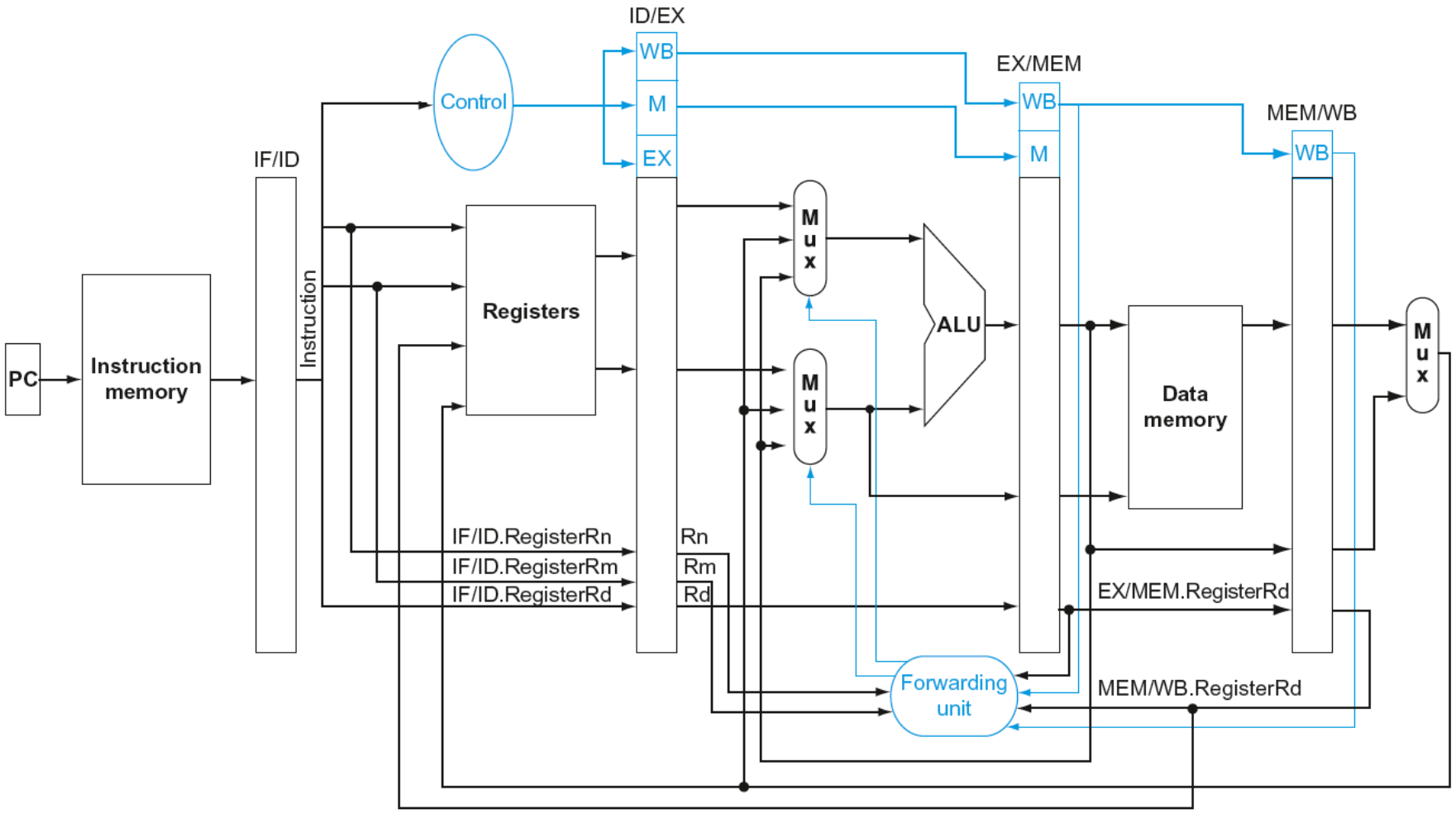
    and (MEM/WB.RegisterRd ≠ 31)

    and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 31)

    and (EX/MEM.RegisterRd ≠ ID/EX.RegisterRm2))

    and (MEM/WB.RegisterRd = ID/EX.RegisterRm2)) ForwardB = 01

Datapath with Forwarding



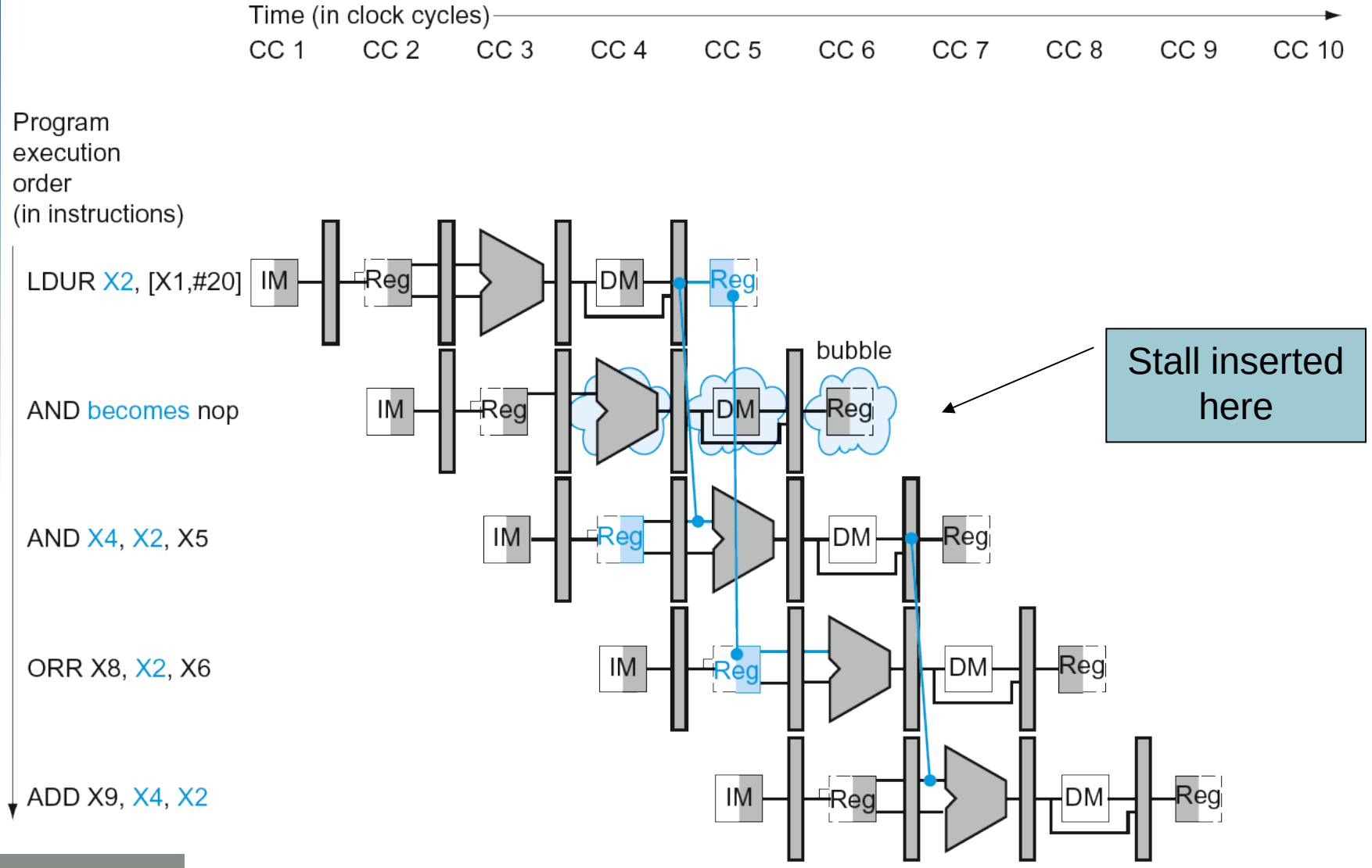
Load-Use Hazard Detection

- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRn1, IF/ID.RegisterRm2
- Load-use hazard when
  - ID/EX.MemRead and
    ((ID/EX.RegisterRd = IF/ID.RegisterRn1) or
    (ID/EX.RegisterRd = IF/ID.RegisterRm1))
- If detected, stall and insert bubble

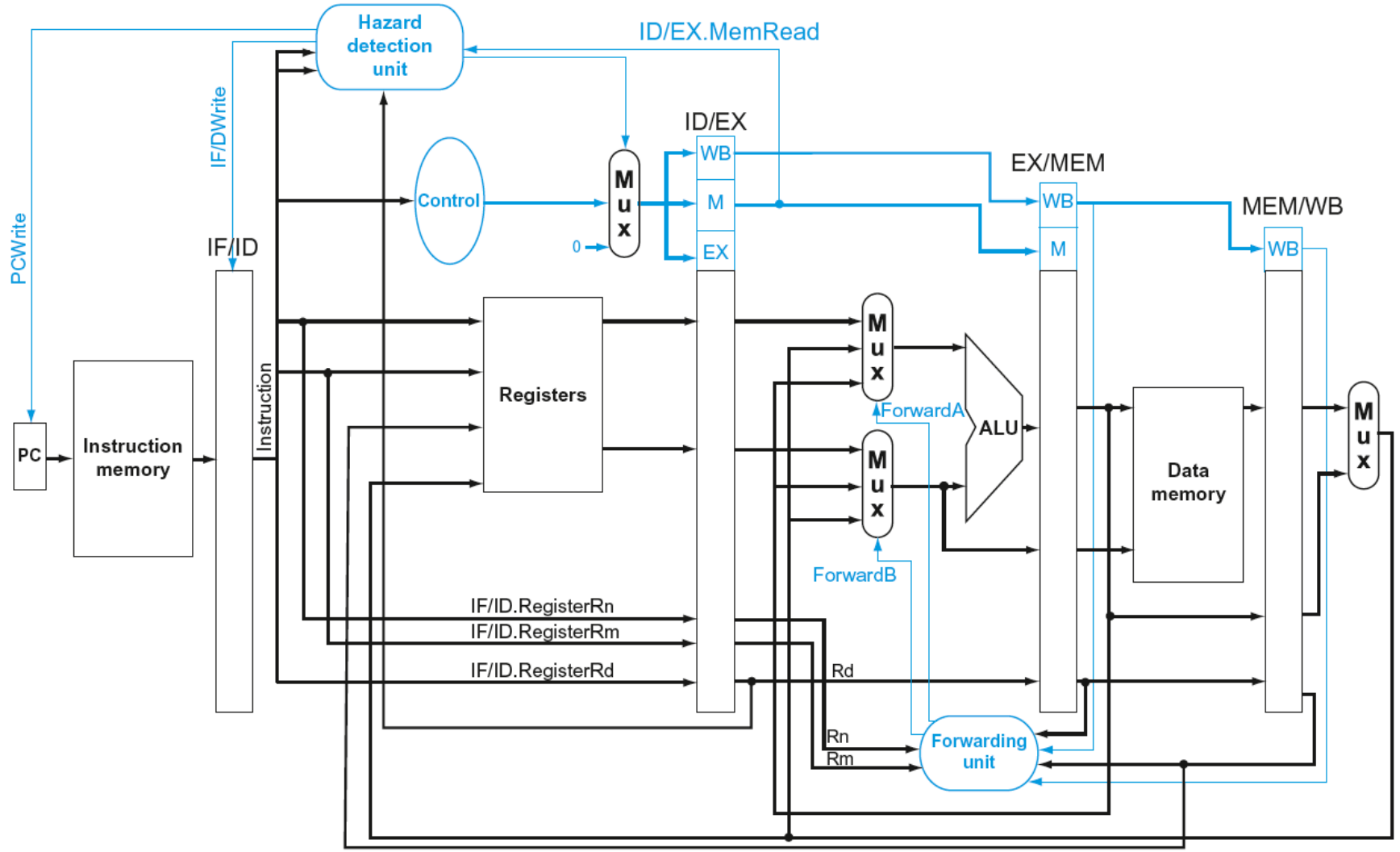
How to Stall the Pipeline

- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for `LDUI`
    - Can subsequently forward to EX stage

Load-Use Data Hazard



Datapath with Hazard Detection



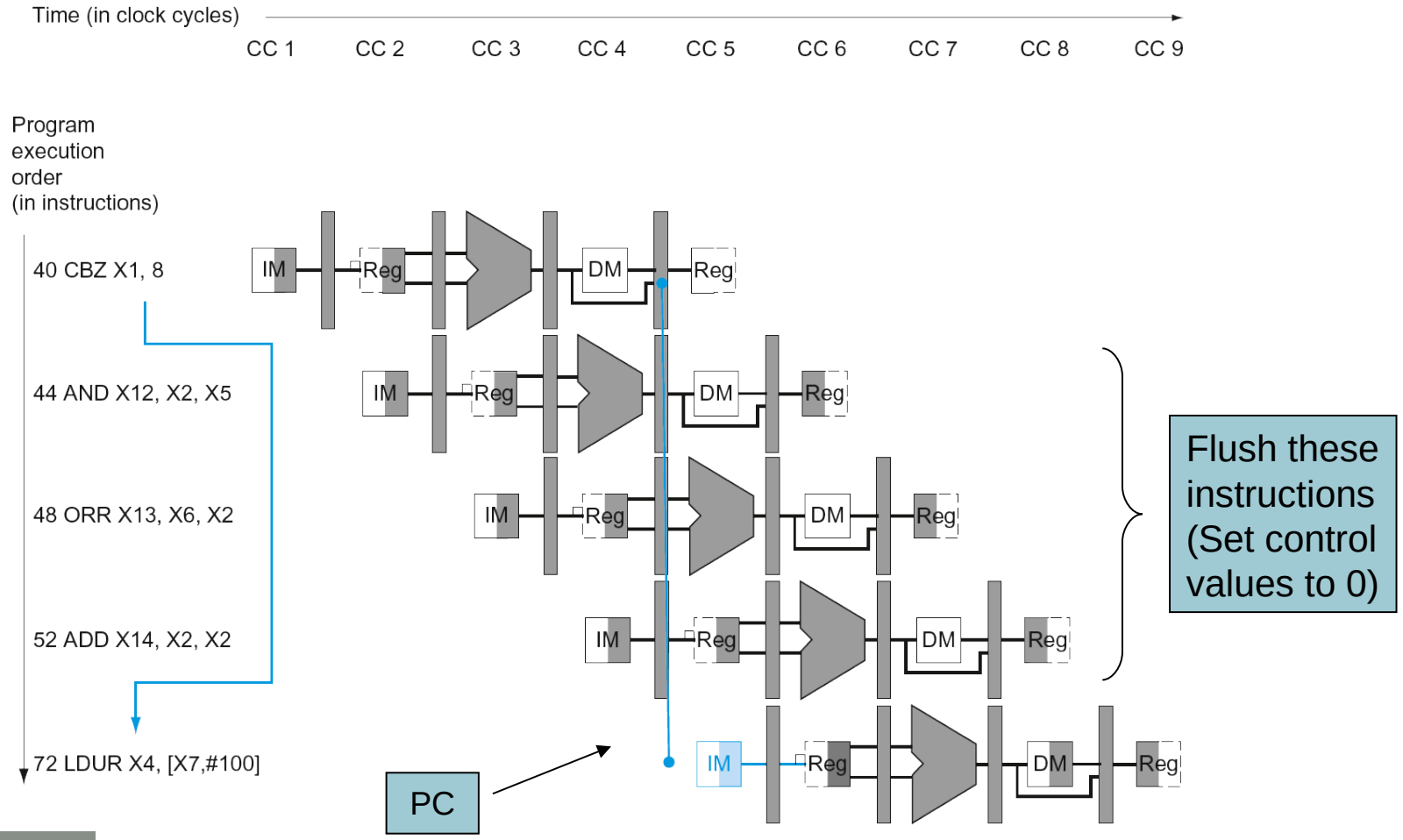
Stalls and Performance

The BIG Picture

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

Branch Hazards

- If branch outcome determined in MEM

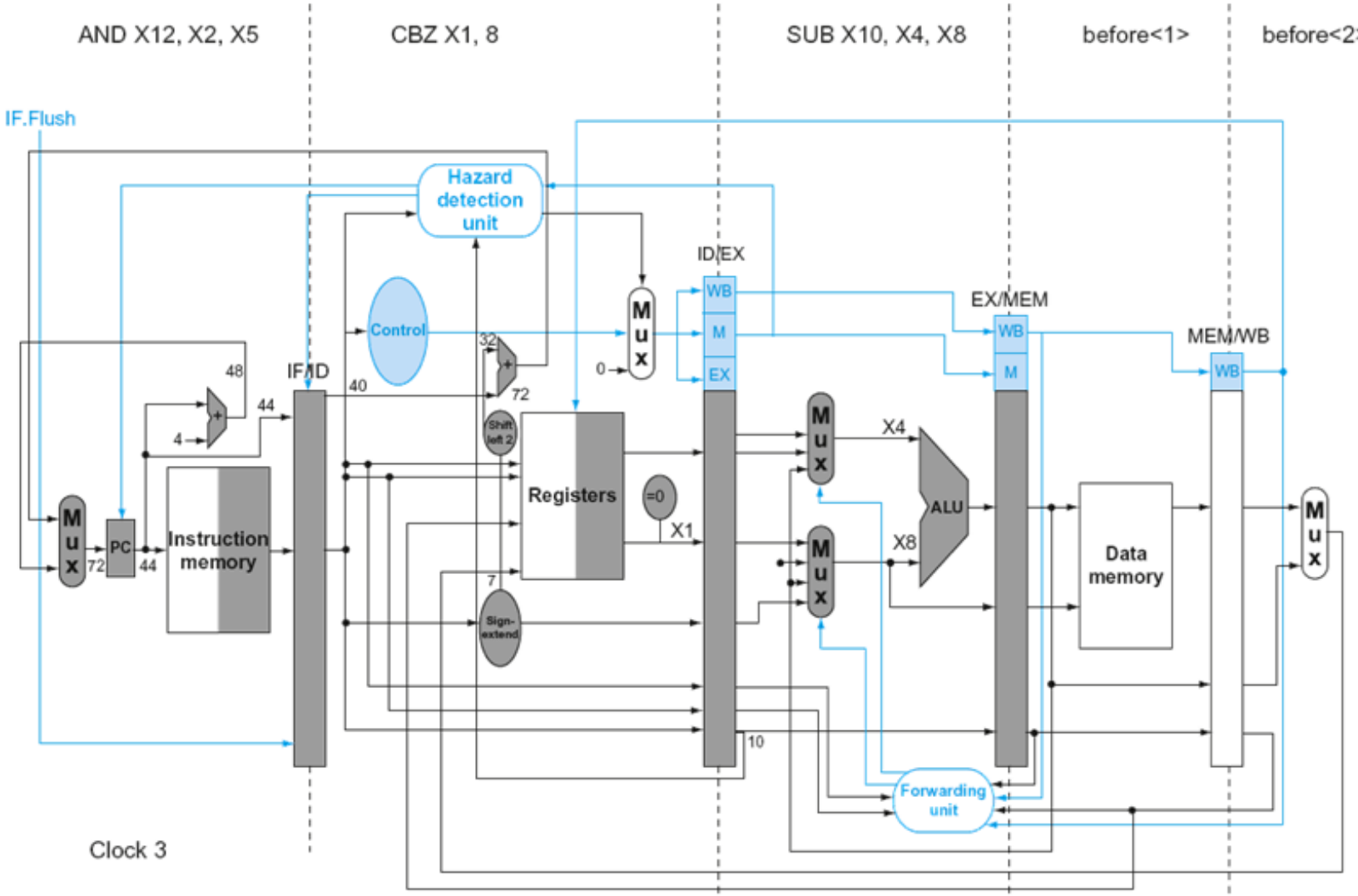


Reducing Branch Delay

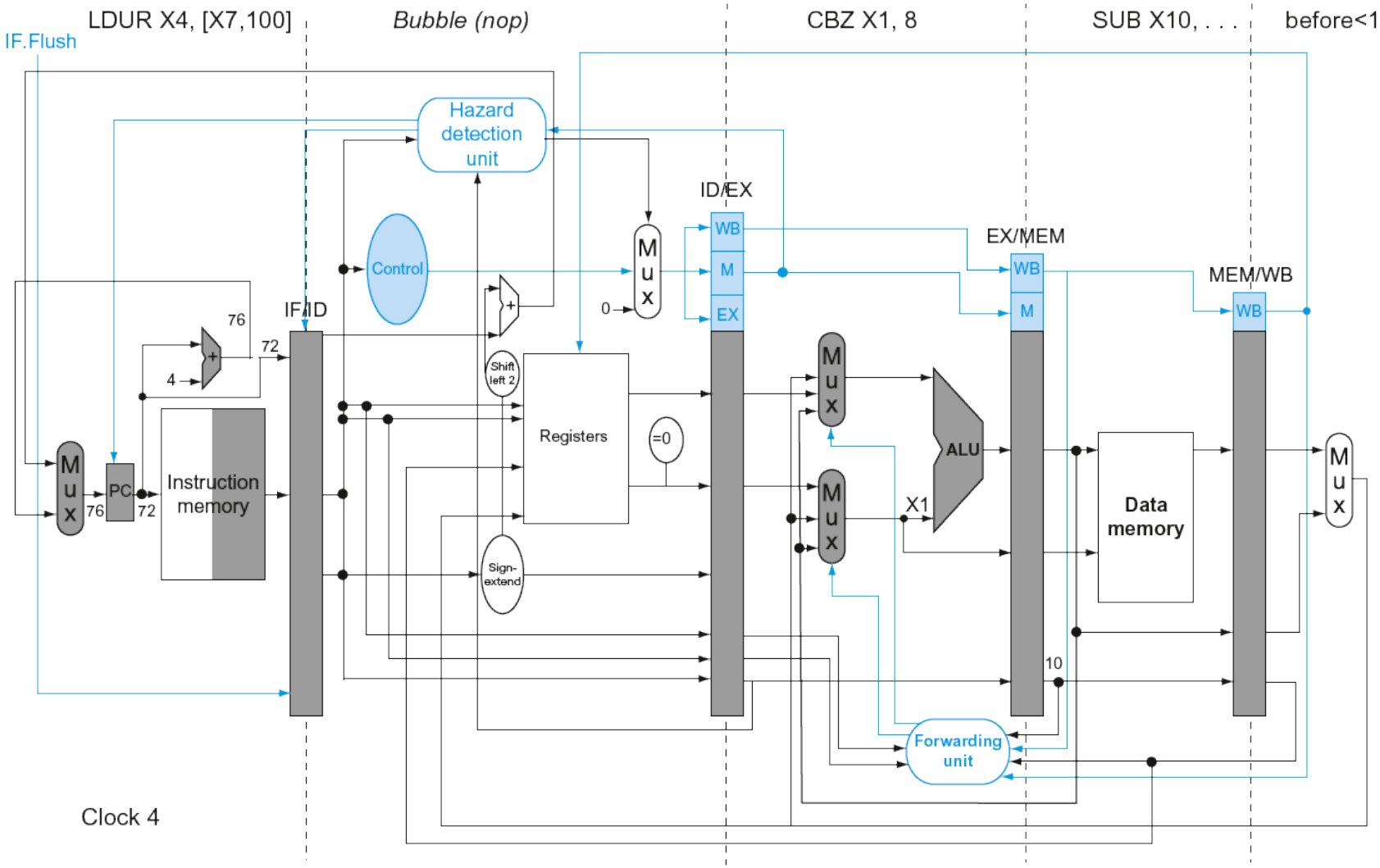
- Move hardware to determine outcome to ID stage
  - Target address adder
  - Register comparator
- Example: branch taken

```
36:  SUB  X10, X4, X8
40:  CBZ  X1,  X3, 8
44:  AND  X12, X2, X5
48:  ORR  X13, X2, X6
52:  ADD  X14, X4, X2
56:  SUB  X15, X6, X7
     ...
72:  LDUR X4, [X7,#50]
```

Example: Branch Taken



Example: Branch Taken

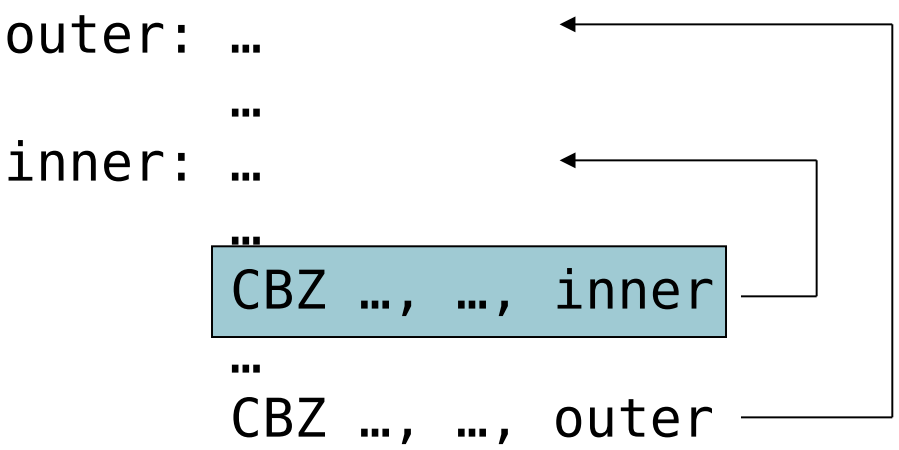


Dynamic Branch Prediction

- In deeper and superscalar pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (aka branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - To execute a branch
    - Check table, expect the same outcome
    - Start fetching from fall-through or target
    - If wrong, flush pipeline and flip prediction

1-Bit Predictor: Shortcoming

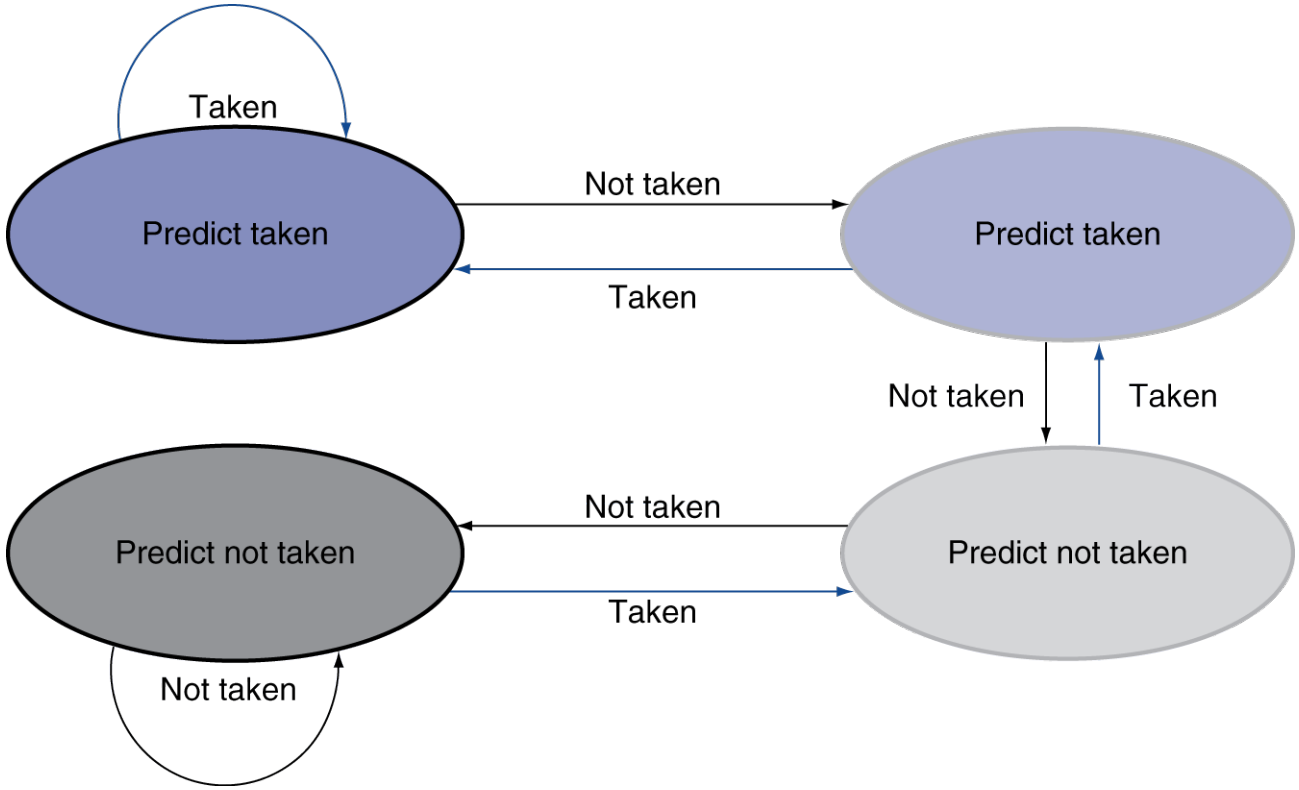
- Inner loop branches mispredicted twice!



  - Mispredict as taken on last iteration of inner loop
  - Then mispredict as not taken on first iteration of inner loop next time around

2-Bit Predictor

- Only change prediction on two successive mispredictions



Calculating the Branch Target

- Even with predictor, still need to calculate the target address
  - 1-cycle penalty for a taken branch
- Branch target buffer
  - Cache of target addresses
  - Indexed by PC when instruction fetched
    - If hit and instruction is branch predicted taken, can fetch target immediately

Ejercicios

- Ejercicio 4:
- Dados los siguientes fragmentos de código

- a) Determinar las dependencias de datos que provoquen *data hazard*. En cada caso indicar: los números de las instrucciones involucradas, en qué etapa se encuentra c/u y el operando en conflicto.
- b) Mostrar el orden de ejecución en un procesador que utilice SOLO la técnica de *stall* para evitar los *data hazard*.
- c) Mostrar el orden de ejecución en un procesador que utilice la técnica de *forwarding y stall*.
- d) ¿Cuántos ciclos toma la ejecución del código en cada caso?
- de instrucciones LEGv8

Ejercicios

- Ejercicio 4:

A	B
1> SUB X6, X1, X3 2> SUB X7, X6, X5	1> ADDI X10, X1, #8 2> LDUR X3, [X10, #0] 3> SUB X3, X2, X10

Ejercicios

- Ejercicio 4 A:

			1	2	3	4	5	6
1	SUB X6,X1,X3		F	D	E	M	W	
2	SUB X7,X6,X5			F	D	E	M	W

Ejercicios

- Ejercicio 5:

- Dado el siguiente fragmento de código de instrucciones LEGv8:
-
- 1> LDUR X1, [X10, #0]
- 2> LDUR X2, [X10, #8]
- 3> ADD X3, X1, X2
- 4> STUR X3, [X10, #24]
- 5> LDUR X4, [X10, #16]
- 6> ADD X5, X1, X4
- 7> STUR X5, [X10, #32]

Ejercicios

- Ejercicio 5:

- a) Analizar en el código las dependencias de datos y determinar cuales generan *data hazards.* En cada caso indicar: los números de las instrucciones involucradas, en qué etapa se encuentra c/u y el operando en conflicto.
- b) Mostrar el orden de ejecución de las instrucciones utilizando un procesador con *forwarding stall*.
- c) Reescribir la sección de código alterando el orden de las instrucciones para evitar *stalls* innecesarios. Mostrar el nuevo orden de ejecución.
- d) ¿Cuántos ciclos toma la ejecución del código en cada caso?

Exceptions and Interrupts

- “Unexpected” events requiring change in flow of control
  - Different ISAs use the terms differently
- Exception
  - Arises within the CPU
    - e.g., undefined opcode, overflow, syscall, …
- Interrupt
  - From an external I/O controller
- Dealing with them without sacrificing performance is hard

Handling Exceptions

- Save PC of offending (or interrupted) instruction
  - In LEGv8: Exception Link Register (ELR)
- Save indication of the problem
  - In LEGv8: Exception Syndrome Rregister (ESR)
  - We'll assume 1-bit
    - 0 for undefined opcode, 1 for overflow

An Alternate Mechanism

- Vectored Interrupts
  - Handler address determined by the cause
- Exception vector address to be added to a vector table base register:
  - Unknown Reason: $00\ 0000_{two}$
  - Overflow: $10\ 1100_{two}$
  - …: $11\ 1111_{two}$
- Instructions either
  - Deal with the interrupt, or
  - Jump to real handler

Handler Actions

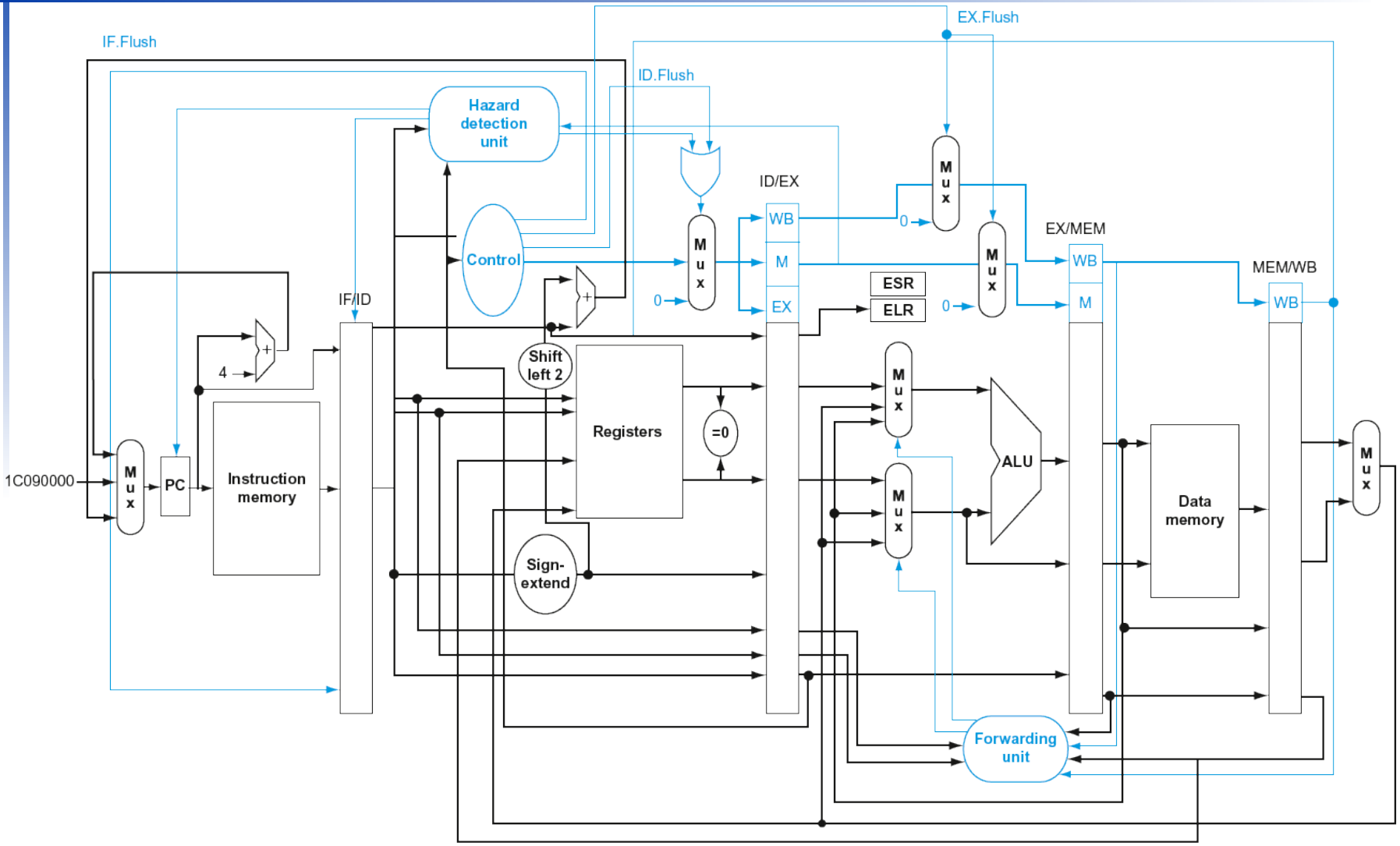
- Read cause, and transfer to relevant handler
- Determine action required
- If restartable
  - Take corrective action
  - use EPC to return to program
- Otherwise
  - Terminate program
  - Report error using EPC, cause, …

Exceptions in a Pipeline

- Another form of control hazard
- Consider overflow on add in EX stage

  `ADD X1, X2, X1`
  - Prevent X1 from being clobbered
  - Complete previous instructions
  - Flush add and subsequent instructions
  - Set ESR and ELR register values
  - Transfer control to handler
- Similar to mispredicted branch
  - Use much of the same hardware

Pipeline with Exceptions



Exception Properties

- Restartable exceptions
  - Pipeline can flush the instruction
  - Handler executes, then returns to the instruction
    - Refetched and executed from scratch
- PC saved in ELR register
  - Identifies causing instruction
  - Actually PC + 4 is saved
    - Handler must adjust

Exception Example

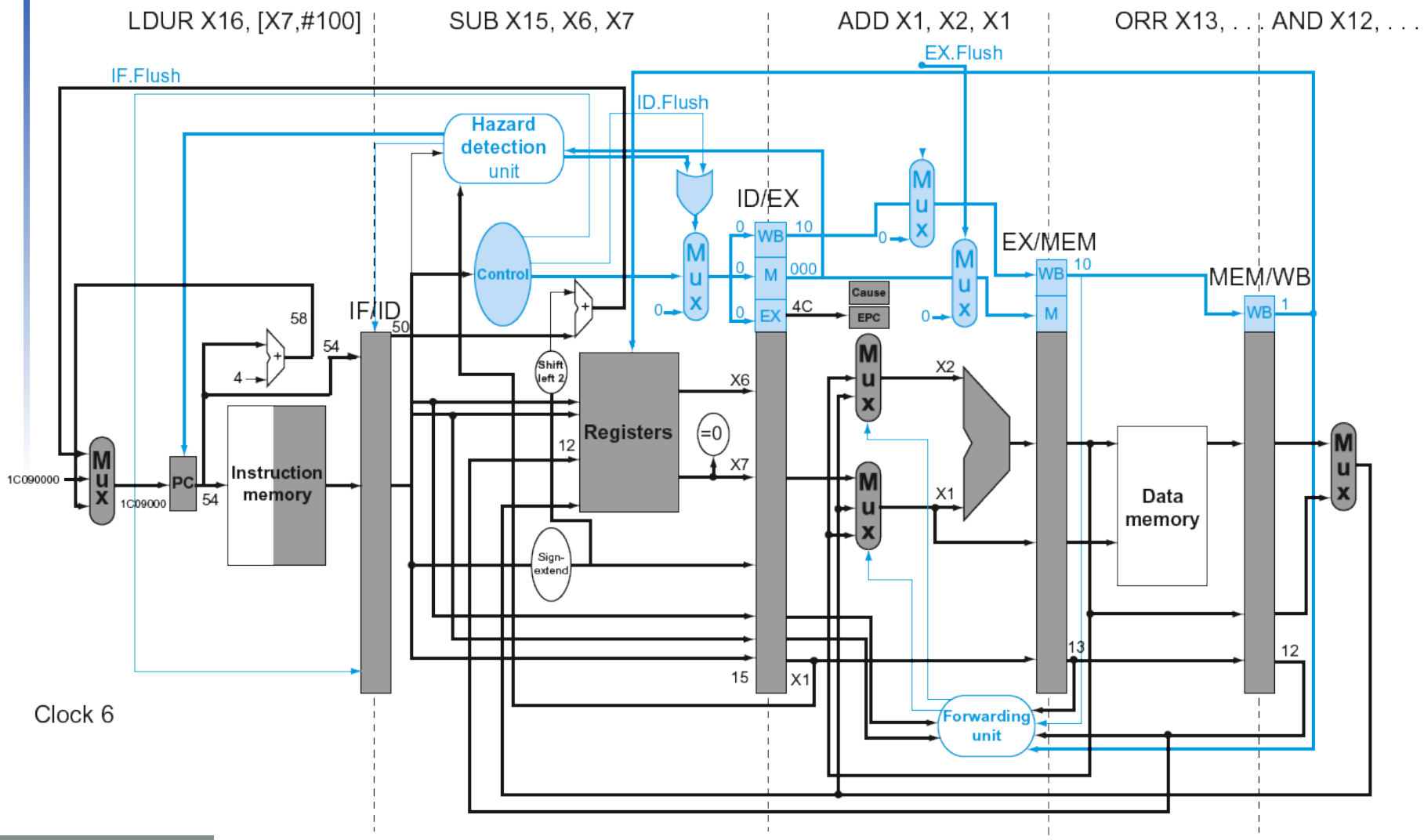
- Exception on ADD in

```
40    SUB  X11, X2, X4
44    AND  X12, X2, X5
48    ORR  X13, X2, X6
4C    ADD  X1,  X2, X1
50    SUB  X15, X6, X7
54    LDUR X16, [X7,#100]
…
```

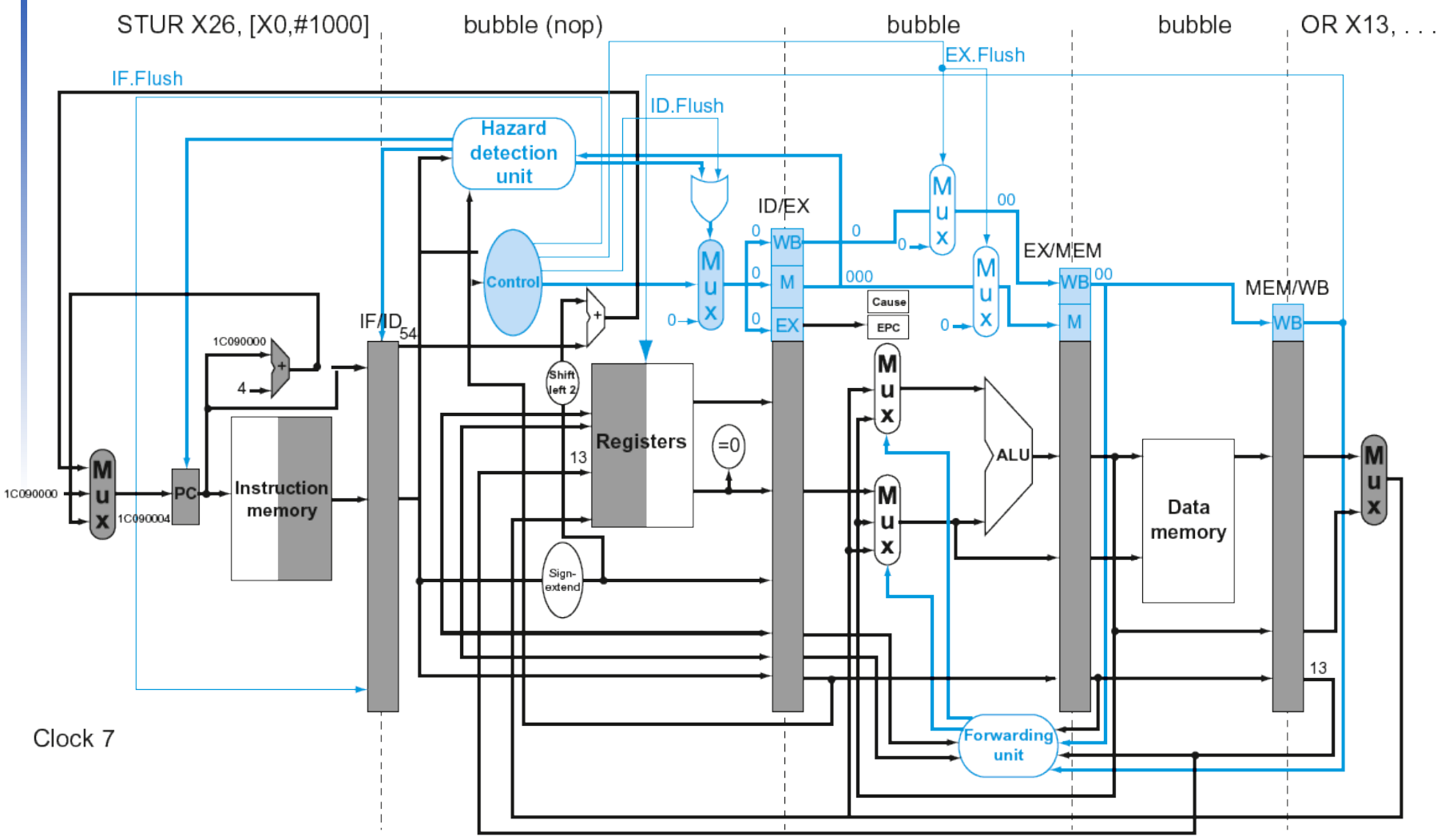
- Handler

```
80000180  STUR X26, [X0,#1000]
80000184   STUR X27, [X0,#1008]
…
```

Exception Example



Exception Example



Multiple Exceptions

- Pipelining overlaps multiple instructions
  - Could have multiple exceptions at once
- Simple approach: deal with exception from earliest instruction
  - Flush subsequent instructions
  - “Precise” exceptions
- In complex pipelines
  - Multiple instructions issued per cycle
  - Out-of-order completion
  - Maintaining precise exceptions is difficult!

Imprecise Exceptions

- Just stop pipeline and save state
  - Including exception cause(s)
- Let the handler work out
  - Which instruction(s) had exceptions
  - Which to complete or flush
    - May require “manual” completion
- Simplifies hardware, but more complex handler software
- Not feasible for complex multiple-issue out-of-order pipelines

Instruction-Level Parallelism (ILP)

- Pipelining: executing multiple instructions in parallel
- To increase ILP
  - Deeper pipeline
    - Less work per stage ⇒ shorter clock cycle
  - Multiple issue
    - Replicate pipeline stages ⇒ multiple pipelines
    - Start multiple instructions per clock cycle
    - CPI < 1, so use Instructions Per Cycle (IPC)
    - E.g., 4GHz 4-way multiple-issue
      - 16 BIPS, peak CPI = 0.25, peak IPC = 4
    - But dependencies reduce this in practice

Multiple Issue

- Static multiple issue
  - Compiler groups instructions to be issued together
  - Packages them into “issue slots”
  - Compiler detects and avoids hazards
- Dynamic multiple issue
  - CPU examines instruction stream and chooses instructions to issue each cycle
  - Compiler can help by reordering instructions
  - CPU resolves hazards using advanced techniques at runtime

Speculation

- “Guess” what to do with an instruction
  - Start operation as soon as possible
  - Check whether guess was right
    - If so, complete the operation
    - If not, roll-back and do the right thing
- Common to static and dynamic multiple issue
- Examples
  - Speculate on branch outcome
    - Roll back if path taken is different
  - Speculate on load
    - Roll back if location is updated

Compiler/Hardware Speculation

- Compiler can reorder instructions
  - e.g., move load before branch
  - Can include “fix-up” instructions to recover from incorrect guess
- Hardware can look ahead for instructions to execute
  - Buffer results until it determines they are actually needed
  - Flush buffers on incorrect speculation

Speculation and Exceptions

- What if exception occurs on a speculatively executed instruction?
  - e.g., speculative load before null-pointer check
- Static speculation
  - Can add ISA support for deferring exceptions
- Dynamic speculation
  - Can buffer exceptions until instruction completion (which may not occur)

Static Multiple Issue

- Compiler groups instructions into “issue packets”
  - Group of instructions that can be issued on a single cycle
  - Determined by pipeline resources required
- Think of an issue packet as a very long instruction
  - Specifies multiple concurrent operations
  - ⇒ Very Long Instruction Word (VLIW)

Scheduling Static Multiple Issue

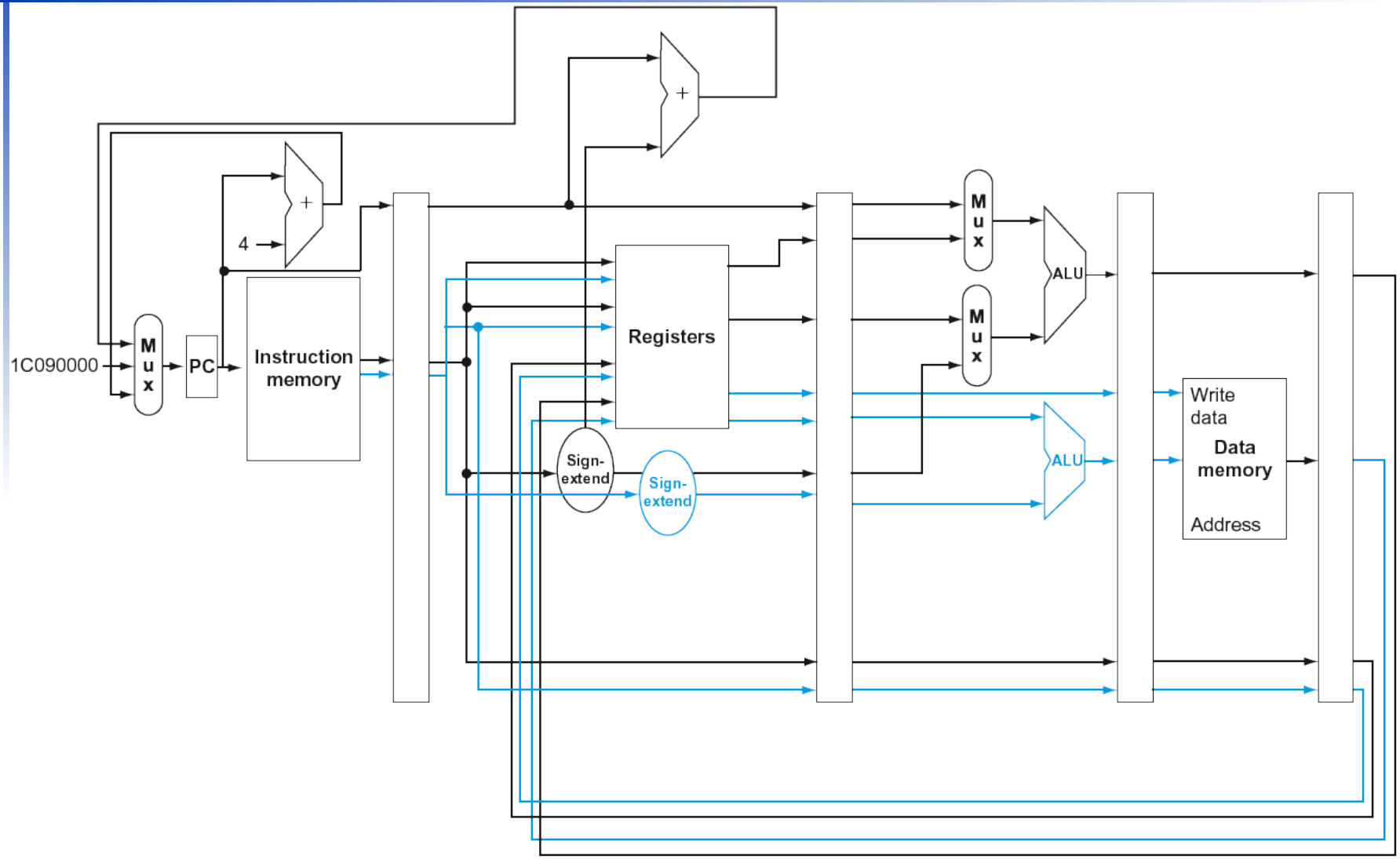
- Compiler must remove some/all hazards
  - Reorder instructions into issue packets
  - No dependencies with a packet
  - Possibly some dependencies between packets
    - Varies between ISAs; compiler must know!
  - Pad with nop if necessary

LEGv8 with Static Dual Issue

- Two-issue packets
  - One ALU/branch instruction
  - One load/store instruction
  - 64-bit aligned
    - ALU/branch, then load/store
    - Pad an unused instruction with nop

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

LEGv8 with Static Dual Issue



Hazards in the Dual-Issue LEGv8

- More instructions executing in parallel
- EX data hazard
  - Forwarding avoided stalls with single-issue
  - Now can’t use ALU result in load/store in same packet
    - ```
      ADD  X0, X0, X1
      LDUR X2, [X0,#0]
      ```
    - Split into two packets, effectively a stall
- Load-use hazard
  - Still one cycle use latency, but now two instructions
- More aggressive scheduling required
      ```

Scheduling Example

- Schedule this for dual-issue LEGv8

```
Loop: LDUR X0, [X20,#0]     // X0=array element
      ADD  X0, X0,X21       // add scalar in X21
      STUR X0, [X20,#0]     // store result
      SUBI X20, X20,#4      // decrement pointer
      CMP  X20, X22         // branch $s1!=0
      BGT Loop
```

	ALU/branch	Load/store	cycle
Loop:	nop	LDUR X0, [X20,#0]	1
	SUBI X20, X20,#4	nop	2
	ADD X0, X0,X21	nop	3
	CMP X20, X22	sw $t0, 4($s1)	4
	BGT Loop	STUR X0, [X20,#0]	5

- IPC = 7/6 = 1.17 (c.f. peak IPC = 2)

Loop Unrolling

- Replicate loop body to expose more parallelism
  - Reduces loop-control overhead
- Use different registers per replication
  - Called “register renaming”
  - Avoid loop-carried “anti-dependencies”
    - Store followed by a load of the same register
    - Aka “name dependence”
      - Reuse of a register name

Loop Unrolling Example

	ALU/branch	Load/store	cycle
Loop:	SUBI X20, X20,#32	LDUR X0, [X20,#0]	1
	nop	LDUR X1, [X20,#24]	2
	ADD X0, X0, X21	LDUR X2, [X20,#16]	3
	ADD X1, X1, X21	LDUR X3, [X20,#8]	4
	ADD X2, X2, X21	STUR X0, [X20,#32]	5
	ADD X3, X3, X21	sw X1, [X20,#24]	6
	CMP X20,X22	sw X2, [X20,#16]	7
	BGT Loop	sw X3, [X20,#8]	8

- IPC = 15/8 = 1.875
  - Closer to 2, but at cost of registers and code size

Dynamic Multiple Issue

- “Superscalar” processors
- CPU decides whether to issue 0, 1, 2, … each cycle
  - Avoiding structural and data hazards
- Avoids the need for compiler scheduling
  - Though it may still help
  - Code semantics ensured by the CPU

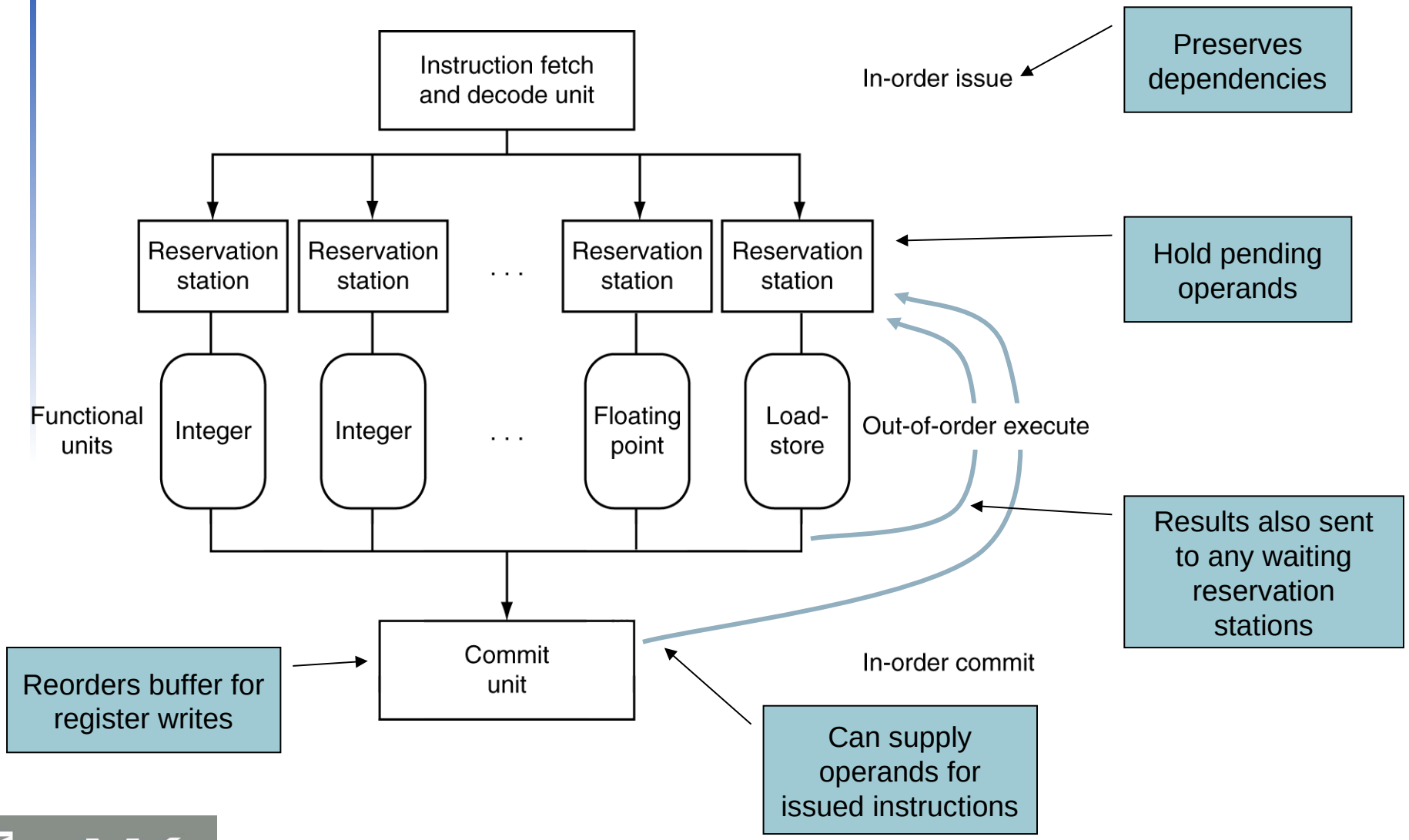
Dynamic Pipeline Scheduling

- Allow the CPU to execute instructions out of order to avoid stalls
  - But commit result to registers in order
- Example

```
LDUR X0, [X21,#20]
ADD  X1, X0, X2
SUB  X23,X23,X3
ANDI X5, X23,#20
```

  - Can start `sub` while `ADD` is waiting for LDUI

Dynamically Scheduled CPU



Register Renaming

- Reservation stations and reorder buffer effectively provide register renaming
- On instruction issue to reservation station
  - If operand is available in register file or reorder buffer
    - Copied to reservation station
    - No longer required in the register; can be overwritten
  - If operand is not yet available
    - It will be provided to the reservation station by a function unit
    - Register update may not be required

Speculation

- Predict branch and continue issuing
  - Don’t commit until branch outcome determined
- Load speculation
  - Avoid load and cache miss delay
    - Predict the effective address
    - Predict loaded value
    - Load before completing outstanding stores
    - Bypass stored values to load unit
  - Don’t commit load until speculation cleared

Why Do Dynamic Scheduling?

- Why not just let the compiler schedule code?
- Not all stalls are predicable
  - e.g., cache misses
- Can’t always schedule around branches
  - Branch outcome is dynamically determined
- Different implementations of an ISA have different latencies and hazards

Does Multiple Issue Work?

The BIG Picture

- Yes, but not as much as we'd like
- Programs have real dependencies that limit ILP
- Some dependencies are hard to eliminate
  - e.g., pointer aliasing
- Some parallelism is hard to expose
  - Limited window size during instruction issue
- Memory delays and limited bandwidth
  - Hard to keep pipelines full
- Speculation can help if done well

Power Efficiency

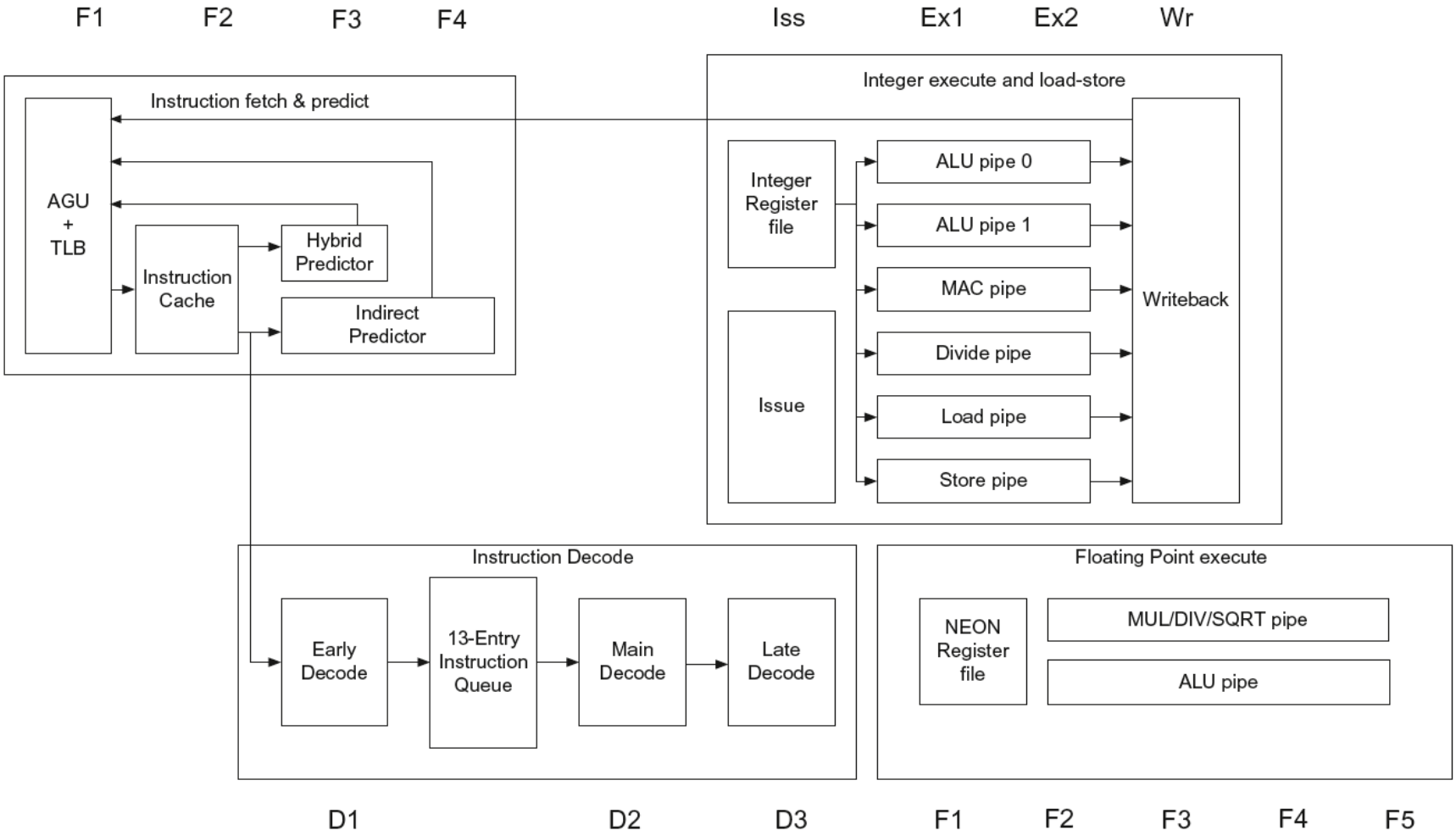
- Complexity of dynamic scheduling and speculations requires power
- Multiple simpler cores may be better

Microprocessor	Year	Clock Rate	Pipeline Stages	Issue width	Out-of-order/ Speculation	Cores	Power
i486	1989	25MHz	5	1	No	1	5W
Pentium	1993	66MHz	5	2	No	1	10W
Pentium Pro	1997	200MHz	10	3	Yes	1	29W
P4 Willamette	2001	2000MHz	22	3	Yes	1	75W
P4 Prescott	2004	3600MHz	31	3	Yes	1	103W
Core	2006	2930MHz	14	4	Yes	2	75W
UltraSparc III	2003	1950MHz	14	4	No	1	90W
UltraSparc T1	2005	1200MHz	6	1	No	8	70W

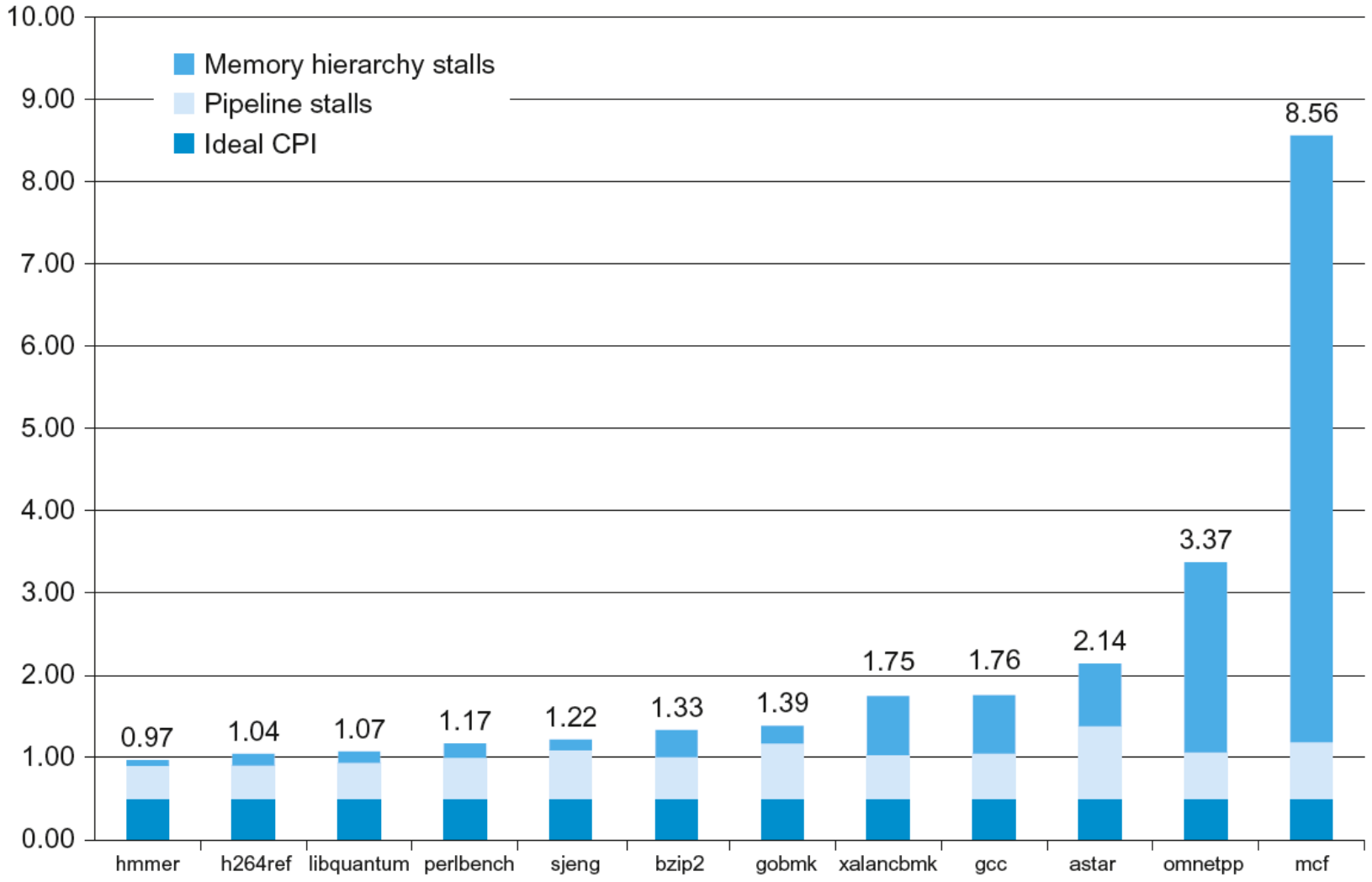
Cortex A53 and Intel i7

Processor	ARM A53	Intel Core i7 920
Market	Personal Mobile Device	Server, cloud
Thermal design power	100 milliWatts (1 core @ 1 GHz)	130 Watts
Clock rate	1.5 GHz	2.66 GHz
Cores/Chip	4 (configurable)	4
Floating point?	Yes	Yes
Multiple issue?	Dynamic	Dynamic
Peak instructions/clock cycle	2	4
Pipeline stages	8	14
Pipeline schedule	Static in-order	Dynamic out-of-order with speculation
Branch prediction	Hybrid	2-level
1st level caches/core	16-64 KiB I, 16-64 KiB D	32 KiB I, 32 KiB D
2nd level caches/core	128-2048 KiB	256 KiB (per core)
3rd level caches (shared)	(platform dependent)	2-8 MB

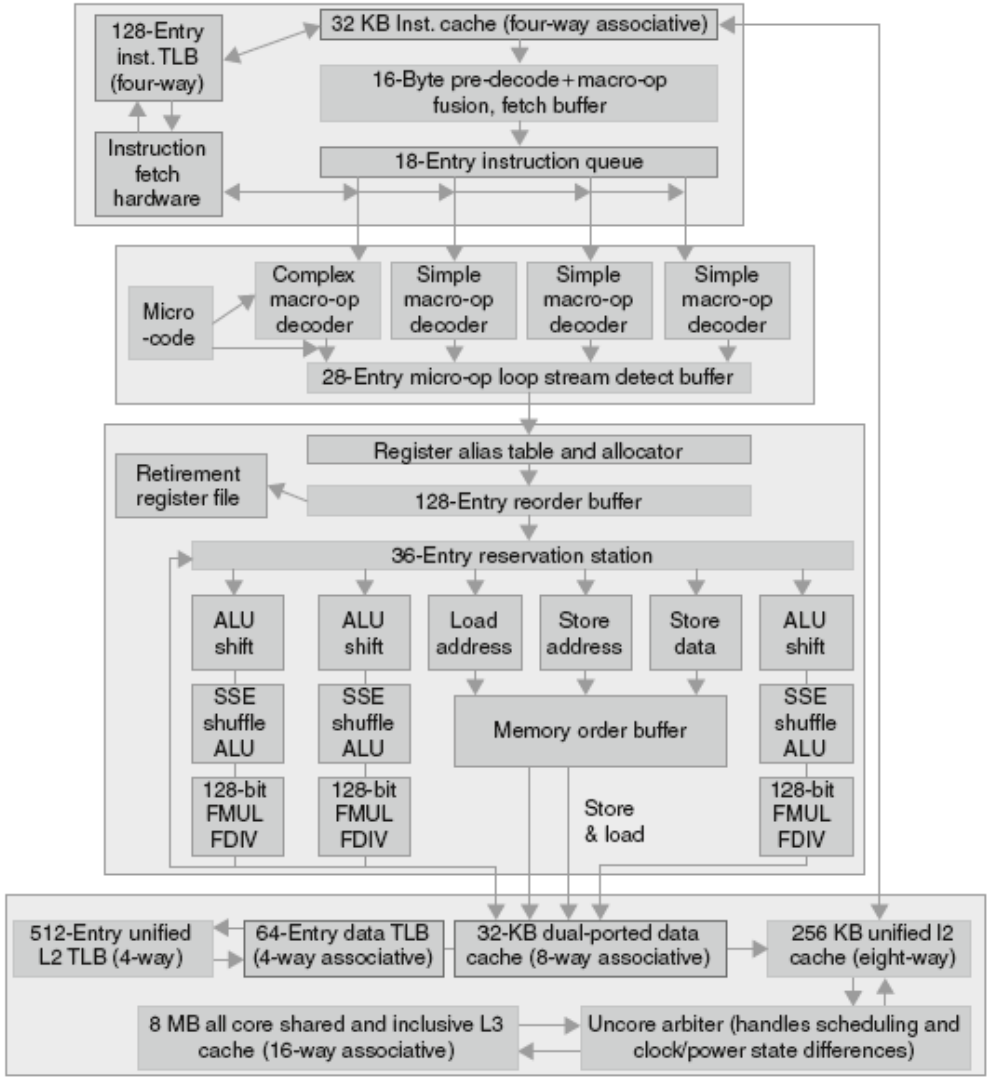
ARM Cortex-A53 Pipeline



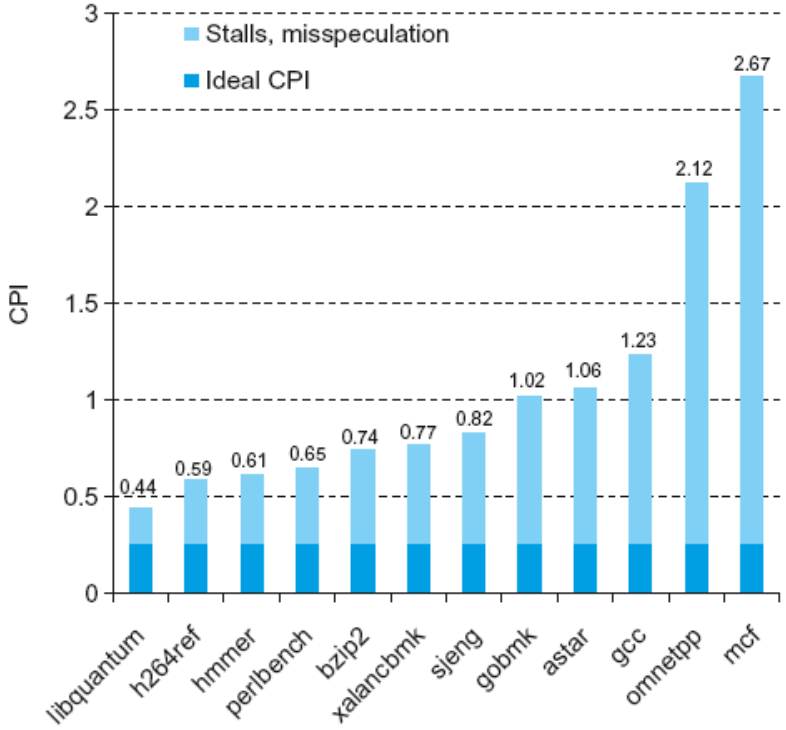
ARM Cortex-A53 Performance

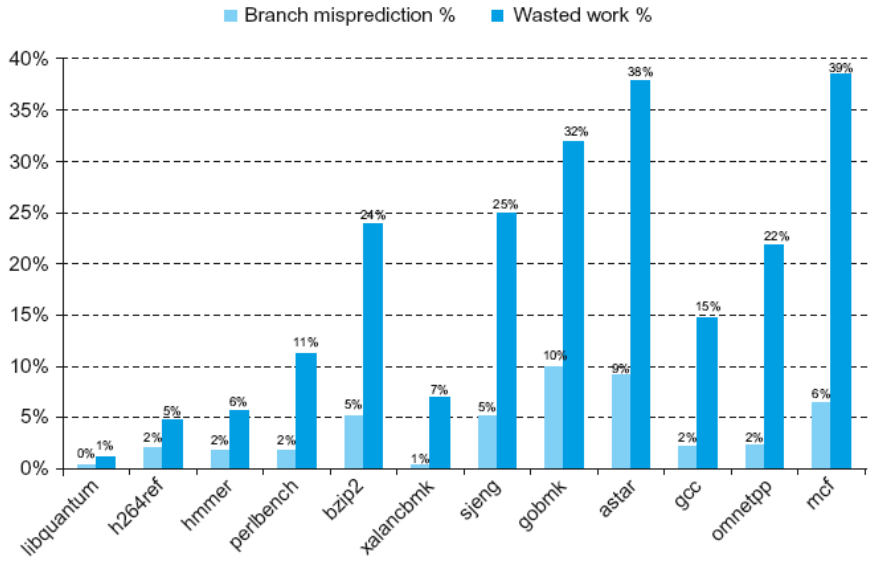


Core i7 Pipeline



Core i7 Performance





Matrix Multiply

- Unrolled C code

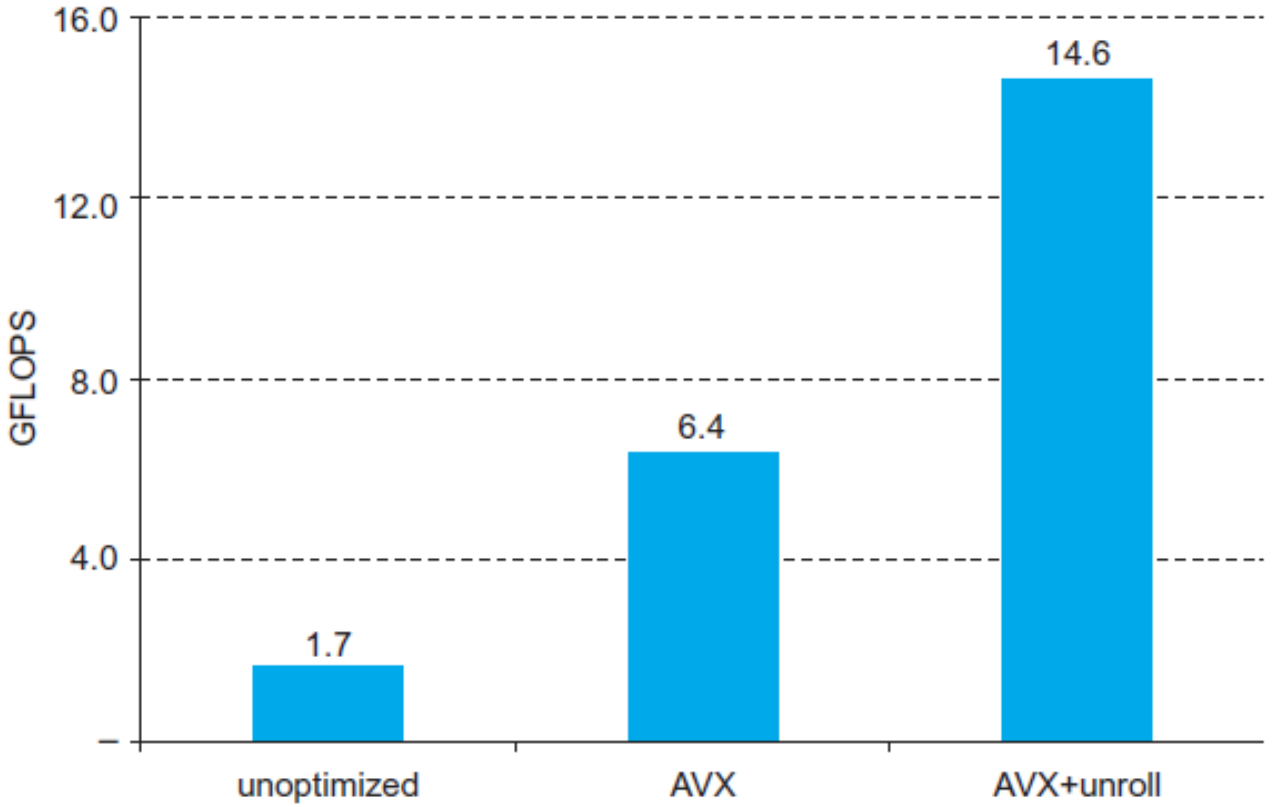
```
1 #include <x86intrin.h>
2 #define UNROLL (4)
3
4 void dgemm (int n, double* A, double* B, double* C)
5 {
6  for ( int i = 0; i < n; i+=UNROLL*4 )
7   for ( int j = 0; j < n; j++ ) {
8    __m256d c[4];
9    for ( int x = 0; x < UNROLL; x++ )
10    c[x] = _mm256_load_pd(C+i+x*4+j*n);
11
12   for( int k = 0; k < n; k++ )
13   {
14    __m256d b = _mm256_broadcast_sd(B+k+j*n);
15    for (int x = 0; x < UNROLL; x++)
16    c[x] = _mm256_add_pd(c[x],
17                         _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
18   }
19
20    for ( int x = 0; x < UNROLL; x++ )
21     _mm256_store_pd(C+i+x*4+j*n, c[x]);
22  }
23 }
```

Matrix Multiply

- Assembly code:

```
1 vmovapd (%r11),%ymm4                # Load 4 elements of C into %ymm4
2 mov %rbx,%rax                       # register %rax = %rbx
3 xor %ecx,%ecx                       # register %ecx = 0
4 vmovapd 0x20(%r11),%ymm3            # Load 4 elements of C into %ymm3
5 vmovapd 0x40(%r11),%ymm2            # Load 4 elements of C into %ymm2
6 vmovapd 0x60(%r11),%ymm1            # Load 4 elements of C into %ymm1
7 vbroadcastsd (%rcx,%r9,1),%ymm0     # Make 4 copies of B element
8 add $0x8,%rcx # register %rcx = %rcx + 8
9 vmulpd (%rax),%ymm0,%ymm5           # Parallel mul %ymm1,4 A elements
10 vaddpd %ymm5,%ymm4,%ymm4           # Parallel add %ymm5, %ymm4
11 vmulpd 0x20(%rax),%ymm0,%ymm5      # Parallel mul %ymm1,4 A elements
12 vaddpd %ymm5,%ymm3,%ymm3           # Parallel add %ymm5, %ymm3
13 vmulpd 0x40(%rax),%ymm0,%ymm5      # Parallel mul %ymm1,4 A elements
14 vmulpd 0x60(%rax),%ymm0,%ymm0      # Parallel mul %ymm1,4 A elements
15 add %r8,%rax                       # register %rax = %rax + %r8
16 cmp %r10,%rcx                      # compare %r8 to %rax
17 vaddpd %ymm5,%ymm2,%ymm2           # Parallel add %ymm5, %ymm2
18 vaddpd %ymm0,%ymm1,%ymm1           # Parallel add %ymm0, %ymm1
19 jne 68 <dgemm+0x68>                # jump if not %r8 != %rax
20 add $0x1,%esi                      # register % esi = % esi + 1
21 vmovapd %ymm4,(%r11)               # Store %ymm4 into 4 C elements
22 vmovapd %ymm3,0x20(%r11)           # Store %ymm3 into 4 C elements
23 vmovapd %ymm2,0x40(%r11)           # Store %ymm2 into 4 C elements
24 vmovapd %ymm1,0x60(%r11)           # Store %ymm1 into 4 C elements
```

Performance Impact



Fallacies

- Pipelining is easy (!)
  - The basic idea is easy
  - The devil is in the details
    - e.g., detecting data hazards
- Pipelining is independent of technology
  - So why haven't we always done pipelining?
  - More transistors make more advanced techniques feasible
  - Pipeline-related ISA design needs to take account of technology trends
    - e.g., predicated instructions

Pitfalls

- Poor ISA design can make pipelining harder
  - e.g., complex instruction sets (VAX, IA-32)
    - Significant overhead to make pipelining work
    - IA-32 micro-op approach
  - e.g., complex addressing modes
    - Register update side effects, memory indirection
  - e.g., delayed branches
    - Advanced pipelines have long delay slots

Concluding Remarks

- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced
- Hazards: structural, data, control
- Multiple issue and dynamic scheduling (ILP)
  - Dependencies limit achievable parallelism
  - Complexity leads to the power wall